AVA Test plan

Version 1.1

SESIP-2500101-01

07/11/2025

NXP AW692/AW693/IW623/IW693

Version 1.0

Developer:	NXP USA, Inc.
Sponsor:	NXP USA, Inc.
Security target:	AW692/AW693/IW623/IW693, SESIP Security Target, Revision 1.0, NXP Semiconductors, 7 November 2025.
Certification body:	TrustCB B.V.
ITSEF:	DEKRA Testing and Certification S.A.U.

Prepared by:

DEKRA Testing and Certification S.A.U.
NIF A29507456
Avenida de los Pirineos, 7
Nave 9A
28703, San Sebastián de los Reyes (Madrid)

Telephone: +34 916588314
+34 916238772

Document approved, revised and signed by the Project Manager: Anders Möller

Document history

Version	Date	Approved by	Changes
1.0	02-10-2025	AOM	First version of the report
1.1	07-11-2025	AOM	New public vulnerability search for the terms of Bluetooth 5.4. In the first version of the report, the ST stated the Bluetooth version was 5.3. After the ST update, the version is 5.4.

CONTENT

1 REFERENCES 6
2 VULNERABILITY ANALYSIS AND PENETRATION TEST PLAN 7
2.1 Introduction 7
3 VULNERABILITIES CATEGORIZATION 8
3.1 [FW] Firmware Attacks 8
3.2 [SW] Software Attacks 8
3.3 [COM] Communication-based Attacks 8
3.4 [CRY] Cryptographic attacks 9
3.5 [MEM] Memory-based Attacks 9
3.6 [LOG] Logical Security Attacks 9
3.7 [INT] External Interface Attacks 9
3.8 [SCA] Side-Channel Attacks 9
3.9 [POW] Fault Injection Attacks 10
3.10 [PHY] Physical Attacks 10
3.11 [MISC] Miscellaneous 10
4 TOE SCOPE AND COMPONENTS 11
5 TOE INTERFACES MAPPING 13
6 PUBLIC VULNERABILITIES 15
6.1 Public vulnerabilities summary 16
6.2 Public vulnerabilities search 16
7 CODE ANALYSIS 23
7.1 Motivation 23
7.2 Test purpose 23
7.3 Tested sample 23
7.4 Description 23
7.5 Results 27
8 VULNERABILITY ANALYSIS 39

8.1 Identity 41

8.2 Secure Initialization 42

8.3 Firmware Update 51

8.4 Countermeasures and Physical Attacks 53

8.5 Secure Communication Support – RCP-Network 60

8.6 Secure Communication Support – RCP-Host 67

8.7 System LifeCycle and Debug Authentication 83

9 PENETRATION TESTING 91

9.1 Introduction 91

10 EVIDENCE LIST 101

1 References

[GP_FST_070]	Security Evaluation Standard for IoT Platforms (SESIP) Version 1.2
[CC31p2]	Common Criteria for Information Technology Security Evaluation. Part 2: Security Functional Components Version 3.1, Revision 5
[CC31p3]	Common Criteria for Information Technology Security Evaluation. Part 3: Security Assurance Components Version 3.1, Revision 5
[CEM31]	Common Criteria for Information Technology Security Evaluation. Evaluation Methodology Version 3.1 Revision
[NXP_AW692-EL_v1.1]	Evidences list, v1.1, 7th November 2025

2 Vulnerability Analysis and Penetration Test Plan

2.1 Introduction

1 The vulnerability analysis and penetration test plan aim to assess the security robustness of the TOE by identifying potential vulnerabilities and weaknesses. This activity outlines the approach to analyse and test the system's security and ability to withstand various penetration tests and ensures that appropriate measures are in place to mitigate risks.

2 As part of this process, analysis of publicly known vulnerabilities has been carried out to identify existing risks. A static source code analysis has been performed followed by a vulnerability analysis of relevant areas of the TOEs security specification and implementation. Based on this analysis, a selection of penetration tests has been defined to be carried out to assess the security of the system and generate evidence to its ability to defend against these attacks.

3 Vulnerabilities Categorization

3 A categorization of vulnerabilities is presented in this section, that will assist to the performance of:

- A systematic search about the technology applicable to the product in public sources.
- The development of an analysis applying jointly the results of the search in public sources and the revision of the evaluation evidence: guidance documents, functional specification, TOE design, security architecture and implementation.
- The design of penetration tests based on the detected vulnerabilities.

3.1 [FW] Firmware Attacks

4 **Firmware Modification**: Altering the firmware to include malicious code or change its behaviour, exploiting vulnerabilities in the firmware update or validation process.

5 **Backdoor Exploitation**: Using backdoors that were intentionally or accidentally introduced into the firmware.

6 **Bootloader Exploits**: Attacks targeting the SoC's bootloader to execute unauthorized code or bypass firmware integrity checks.

3.2 [SW] Software Attacks

7 **Buffer Overflows**: Exploiting buffer overflow vulnerabilities in software running on the SoC to gain control or cause security failures.

8 **Kernel Exploits**: Attacks that target vulnerabilities in the SoC's operating system kernel.

9 **Rootkits/Bootkits**: Installing malicious software at the kernel or bootloader level to maintain persistent, hidden access to the system.

3.3 [COM] Communication-based Attacks

10 **Attacks on Communication Protocols**: Exploiting vulnerabilities in the protocols used for communication between the SoC and other components, such as Wi-Fi, Bluetooth, I2C, SPI, UART, etc.

11 **Man-in-the-Middle (MitM)**: Intercepting and manipulating communications between the SoC and other devices to gain sensitive data or alter behaviour.

3.4 [CRY] Cryptographic attacks

12 **Cryptographic Attacks**: Attacks that aim to weaken or break cryptographic implementations, like brute-force attacks or exploiting weak key generation.

3.5 [MEM] Memory-based Attacks

13 **Cold Boot Attacks**: Extracting data from the SoC's non-volatile and volatile memory after a reboot or power-off.

14 **Rowhammer**: Manipulating DRAM memory by repeatedly accessing adjacent memory rows, inducing bit flips in nearby rows.

3.6 [LOG] Logical Security Attacks

15 **Trusted Execution Environment (TEE) Exploitation**: Attacks targeting trusted execution environments like ARM TrustZone to compromise their security and access sensitive data.

16 **Secure Boot Attacks**: Attempting to bypass or compromise the secure boot process to get the SoC to run unauthorized code.

17 **Secure Update Attacks:** Attempting to bypass or compromise the secure update process to get the SOC to run unauthorized code.

3.7 [INT] External Interface Attacks

18 **Debug Port Exploitation**: Abusing debug ports (such as JTAG or SWD) to gain unauthorized access to the SoC.

19 **API Interface Attacks**: Exploiting bugs or weak configurations in the SoC's APIs used to interact with its software or hardware.

3.8 [SCA] Side-Channel Attacks

20 **Timing Attacks**: Using precise measurements of the time taken to perform cryptographic operations to deduce sensitive information.

21 **Cache Attacks**: Exploiting how data is handled in cache memory to leak information.

22 **Power leakage attacks**: Use precise measurements and statistical methods to determine e.g. cryptographic keys based on measurements in consumed power.

23 **Electromagnetic leakage attacks**: Use precise measurements and statistical methods to determine e.g. cryptographic keys based on measurements of electromagnetic emissions

3.9 [POW] Fault Injection Attacks

24 **Clock Glitching**: Manipulation of the device clock signal. Aims to insert a too-narrow, or too-wide, clock edge, to induce operational errors leading to attacks on security functions.

25 **Voltage Glitching**: Manipulating the SoC's voltage to induce operational errors leading to attacks on security functions.

26 **Electromagnetic (EM) Fault Injection**: Emission of electromagnetic pulses on a specific location of the device can induce a fault leading to attacks on security functions.

3.10 [PHY] Physical Attacks

27 **Physical Attacks**: Direct manipulation of hardware, such as chip decapsulation or pin tampering to alter the system's behaviour.

3.11 [MISC] Miscellaneous

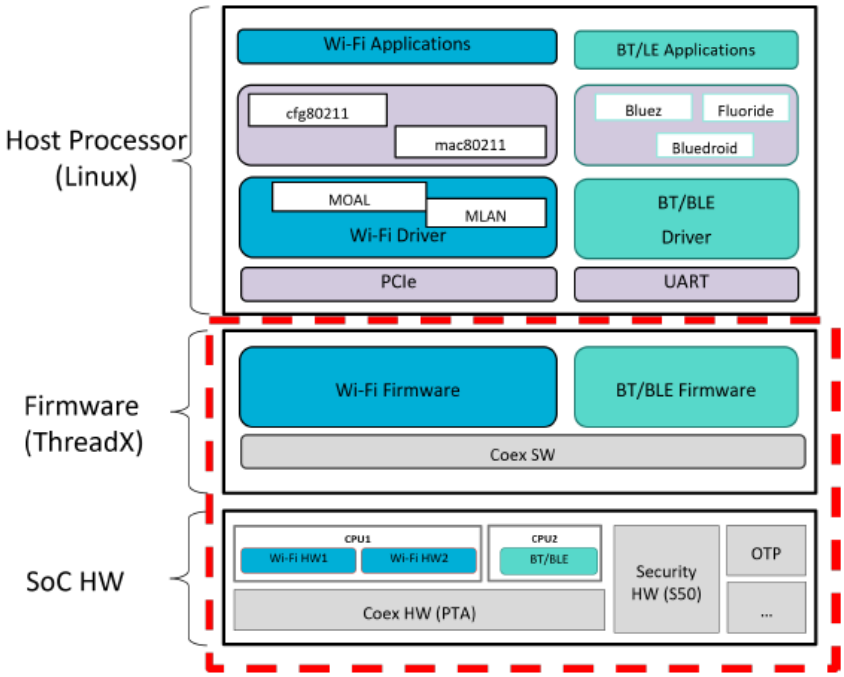
28 Any attack, which is not included in any of the previous categories or can be included in multiple categories, will be included in this class of the typology.

4 TOE scope and components

29 The scope of the evaluation and components included in the TOE are the main points that condition the target's vulnerability analysis.

30 This information is mainly included in the security target [ST_10], which the evaluator has consulted for the necessary information.

31 From the section 1.5.1 Platform Security Features and scope of the [ST_10], the figure 1 regarding the TOE evaluation scope can be seen:



32

33 On the other hand, the table 4 includes the hardware/software components and interfaces:

Components	Hardware components/interfaces	Software components/interfaces
WiFi	CPU 1 - Arm Cortex R7 and dedicated hardware (e.g. memories) Communication port WiFi 6E 5-7GHz / WiFI 6 2.4GHz Host interfaces: PCIe 1.0	BootROM and FW APIs 802.11 ax protocol APIs PCIe protocol APIs
BT/BLE	CPU 2 - Arm Cortex M3 and dedicated hardware (e.g. memories) Communication port Bluetooth 5.3 and BLE Host interfaces: UART1 Digital audio interfaces: I2S, PCM	BootROM and FW APIs BT and BLE protocol APIs UART protocol APIs I2S and PCM interfaces
Coex	Coex Hardware GPIOs+	WCI-2, PTA signals
Others	OTP controller	

34

35 Based on the above information, and the support of the document [AW/IW693_SAM], the evaluator has carried out the study of the attack scenarios applicable to the TOE.

5 TOE interfaces mapping

36 Based on ADV assurance activities, the manufacturer has provided a table that includes the mapping of each of the TOE interfaces with the corresponding SFR defined in [ST_10].

37 In addition, the same table mentioned above also includes the mapping with the code module provided by the corresponding manufacturer. This table is shown below:

SFR	TSFIs	Code module
Verification of Platform Identity	uAP (Wi-Fi), HCI (BT)	-OTP code included in: "aw693w-a1-bootrom-src/common" -Secure bootloader code included in: "aw693w-a1-bootrom-src/common/seboot"
Verification of Platform Instance Identity	PCIe, UART	-OTP code included in: "aw693w-a1-bootrom-src/common" -Secure bootloader code included in: "aw693w-a1-bootrom-src/common/seboot"
Secure Initialization of Platform	PCIe, UART	General Secure bootloader code included in: "aw693w-a1-bootrom-src/common/seboot" CPU 1 bootcode included in: "aw693w-a1-bootrom-src/cpu1" CPU 2 bootcode included in: "aw693w-a1-bootrom-src/cpuX"
Secure Update of Platform	PCIe, UART	The Secure Update functionality is included in the secure boot flow: -General Secure bootloader code included in: "aw693w-a1-bootrom-src/common/seboot" Anti-rollback mechanism:

		-Reference version stored in OTP: "aw693w-a1-bootrom-src/common"
Physical Attacker Resistance	PCIe, UART	Glitch Detector code included in: "AW693_security_related_source_code /GDET" Hardening for general bootcode include in: "aw693w-a1-bootrom-src/common"
Secure Communication Support – RCP-Host	PCIe	NanoTLS through Bluetooth code included in: "AW693_security_related_source_code /bt_sec_if/" NanoTLS through WiFi code included in: "AW693_security_related_source_code /wifi_sec_if"
Secure Communication Support – RCP-Network	SDIO, SPI, UART, USB	Bluetooth code included in: "AW693_security_related_source_code /bt_sec_if/" WiFi code included in: "AW693_security_related_source_code /wifi_sec_if"
Secure Debugging	JTAG	Included in Secure bootloader common code: "aw693w-a1-bootrom-src/common"
Availability Support	PCIe, UART	Watchdog timer code included in: "AW693_security_related_source_code /watchdog_timer Glitch Detector code included in: "AW693_security_related_source_code /GDET"

38 The mapping table with the interfaces constitutes one of several inputs to identify attack scenarios and to define the test plan.

6 Public vulnerabilities

6.1 Public vulnerabilities summary

39 The evaluator has reviewed different sources of public vulnerabilities. The public vulnerabilities in this report have been checked up to October 1, 2025. After the analysis, the evaluator concludes that there are not public vulnerabilities with known exploits related to the TOE until the specified date October 1, 2025.

6.2 Public vulnerabilities search

40 The evaluator has performed a search in the following vulnerability databases in order to perform a deeper and complete vulnerability analysis.

41 In addition, the evaluator has performed a complete scan of the different modules of the source code provided by the manufacturer using the Synopsys tool Black Duck.

42 It has been identified that NXP does not have their own security bulletin.

43 Moreover, the evaluator has searched in following URLs in order to find TOE vulnerabilities:

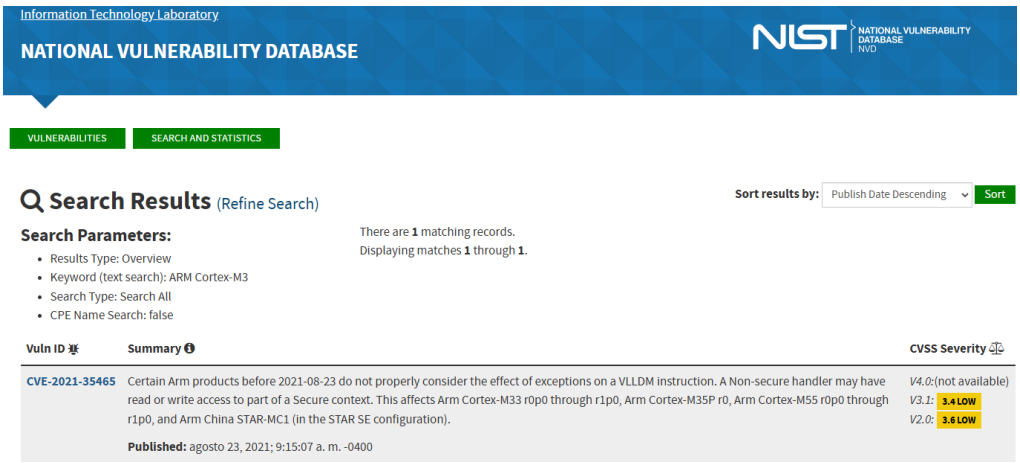
- NIST National Vulnerabilities Database: https://web.nvd.nist.gov/view/vuln/search
- Common Vulnerabilities and Exposures: http://cve.mitre.org/cve/
- CVE Details: https://www.cvedetails.com/vulnerability-search.php
- US-CERT: http://www.kb.cert.org/vuls/html/search/
- Offensive Security Exploit Database: https://www.exploit-db.com/
- Vulnerability Notes Database (VulnDB): https://vuldb.com/es/
- Rapid7 Vulnerability Database: https://www.rapid7.com/db/vulnerabilities
- Github: https://github.com/
- Gitlab: https://about.gitlab.com/

44 The evaluator has performed the search based on the following terms regarding with the TOE:

- ARM Cortex-M3
- ARM Cortex R7
- NXP CLNS
- CSSv2 IP
- AW692
- AW693
- IW623
- IW693
- Wi-Fi 6
- Bluetooth 5.4

45 **NIST National Vulnerabilities Database**

46 The evaluator found 1 new CVE for ARM-Cortex M33.



47

48 CVE found:

- CVE-2021-35465

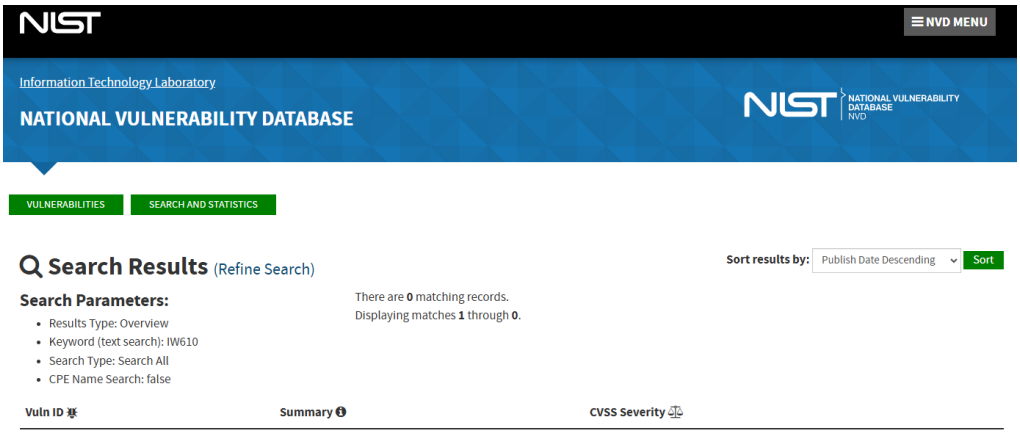
49 This vulnerability affects ARM Cortex-M33, among others, but no to ARM Cortex-M3. Thus, this vulnerability is not applicable to the TOE.

50 The evaluator did not find new CVEs for ARM Cortex R7.

51 The evaluator did not find new CVEs for NXP CLNS.

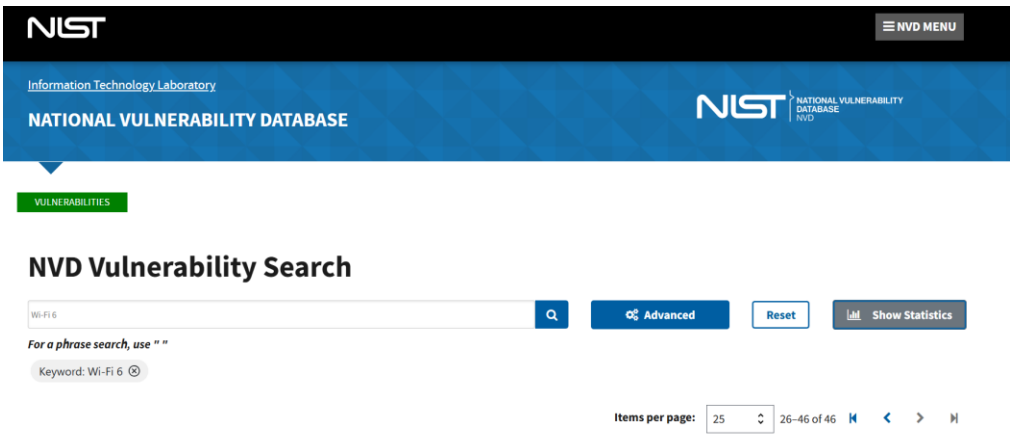
52 The evaluator did not find new CVEs for CSSv2 IP.

53 The evaluator did not find CVEs for AW692, AW693, IW623 and IW693.



54

55 The evaluator found 46 CVEs for Wi-Fi 6.



56

57 Most of the reviewed CVEs affect legacy mobile clients, legacy routers, or third-party software, and are not applicable to Wi-Fi 6 or to the TOE.

58 Only four recent CVEs show vulnerabilities in Wi-Fi 6 devices that could pose a risk in environments similar to the TOE. The remaining CVEs do not directly affect the TOE.

CVE	Risk	Affects the TOE?	Comment
CVE-2024-39963	RCE via parameters (macFilterType)	No	Exploitation requires the host; there is no direct interface for an external attacker

			over PCIe.
CVE-2024-40750	PSK transmitted in cleartext during onboarding	No	Host→RCP traffic is encrypted with NanoTLS; exposure depends on the host, not the TOE.
CVE-2024-53941	PSK derivable from BSSID/MAC	No	Not considered within scope, since the secure management of key material, such as the PSK, is an assumption on the operational environment.
CVE-2025-57579	RCE via default credentials	No	Not considered a TOE risk here because it depends on OEM configuration (default credentials).

59 The evaluator found 5 CVEs for Bluetooth 5.4.

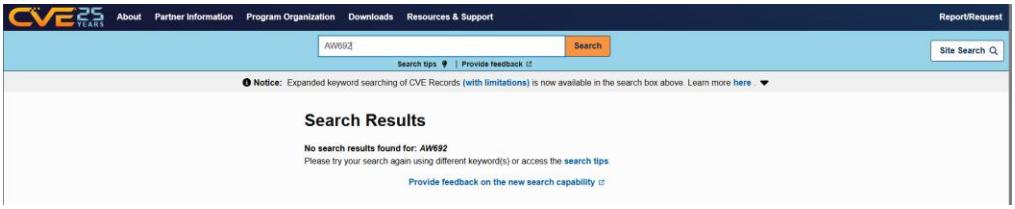
Identifier	CISA Kev Info	Published Date	CNA	Description
CVE-2023-24023	✖	2023-11-28	MITRE	Bluetooth BR/EDR devices with Secure Simple Pairing and Secure Connections pairing in Bluetooth Core Specification 4.2 through 5.4 allow certain man-in-the-middle attacks that force a short key length, and might lead to discovery of the encryption key and live injection, aka BLUFFS.
CVE-2022-48947	✖	2024-10-21	kernel.org	In the Linux kernel, the following vulnerability has been resolved: Bluetooth: L2CAP: Fix u8 overflow By keep sending L2CAP_CONF_REQ packets, chan->num_conf_rsp increases multiple times and eventually it will wrap around the maximum number (i.e., 255). This patch prevents this by adding a boundary...
CVE-2022-48759	✖	2024-06-20	kernel.org	In the Linux kernel, the following vulnerability has been resolved: rpmsg: char: Fix race between the release of rpmsg_ctrldev and cdev struct rpmsg_ctrldev contains a struct cdev. The current code frees the rpmsg_ctrldev struct in rpmsg_ctrldev_release_device(), but the cdev is a managed object, ...
CVE-2019-8922	✖	2021-11-29	MITRE	A heap-based buffer overflow was discovered in bluetoothd in BlueZ through 5.48. There isn't any check on whether there is enough space in the destination buffer. The function simply appends all data passed to it. The values of all attributes that are requested are appended to the output buffer. The...
CVE-2019-8921	✖	2021-11-29	MITRE	An issue was discovered in bluetoothd in BlueZ through 5.48. The vulnerability lies in the handling of a SVC_ATTR_REQ by the SDP implementation. By crafting a malicious CSTATE, it is possible to trick the server into returning more bytes than the buffer actually holds, resulting in leaking arbitrary...
CVE-2017-1000250	✖	2017-09-12	MITRE	All versions of the SDP server in BlueZ 5.46 and earlier are vulnerable to an information disclosure vulnerability which allows remote attackers to obtain sensitive information from the bluetoothd process memory. This vulnerability lies in the processing of SDP search attribute requests.

60

61 There have been no exploits for the CVEs, and thus the evaluator has not considered relevant to specifically target these CVEs. Nevertheless, a detailed fuzzing campaign will be carried out to discover all possible vulnerabilities.

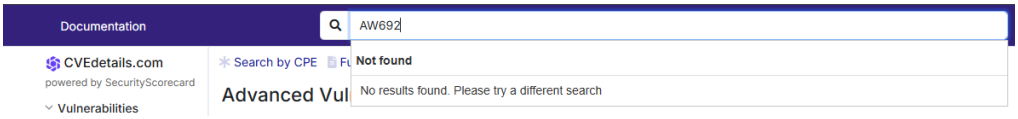
62 **Common Vulnerabilities and Exposures**

63 No vulnerabilities were identified for the terms regarding with the TOE.

64 

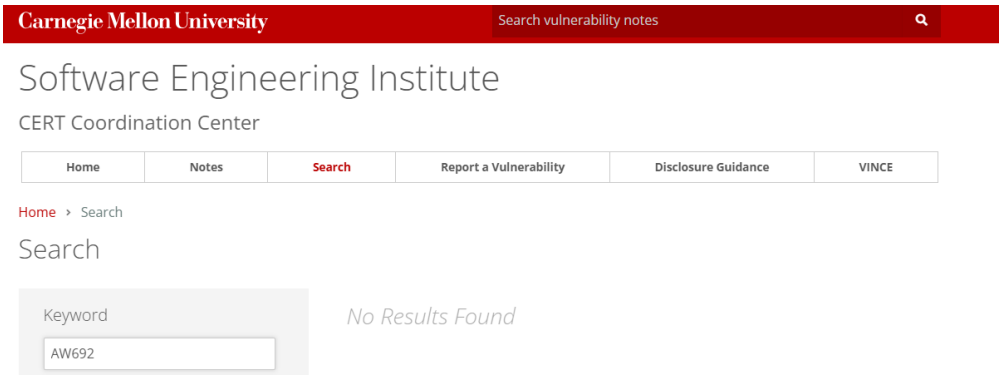
65 **CVE Details**

66 No vulnerabilities were identified for the terms regarding with the TOE.

67 

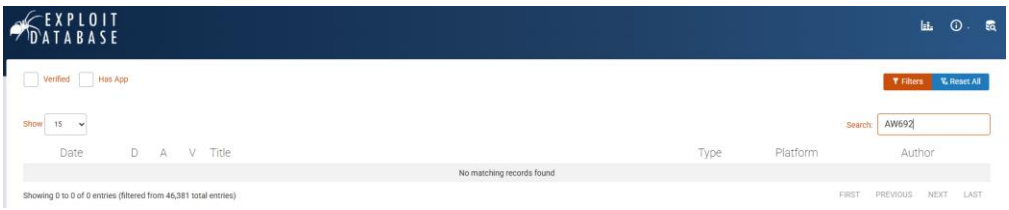
68 **US-CERT**

69 No vulnerabilities were identified for the terms regarding with the TOE.

70 

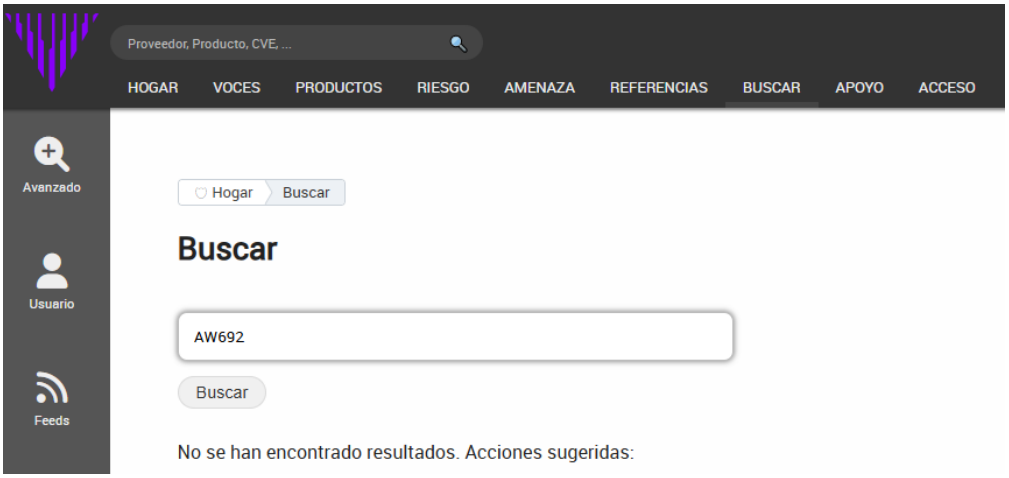
71 **Offensive Security Exploit Database**

72 No vulnerabilities were identified for the terms regarding with the TOE.

73 

74 **Vulnerability Notes Database**

75 No vulnerabilities were identified for the terms regarding with the TOE.

76 

77 **Github**

78 No vulnerabilities were identified for the terms regarding with the TOE.

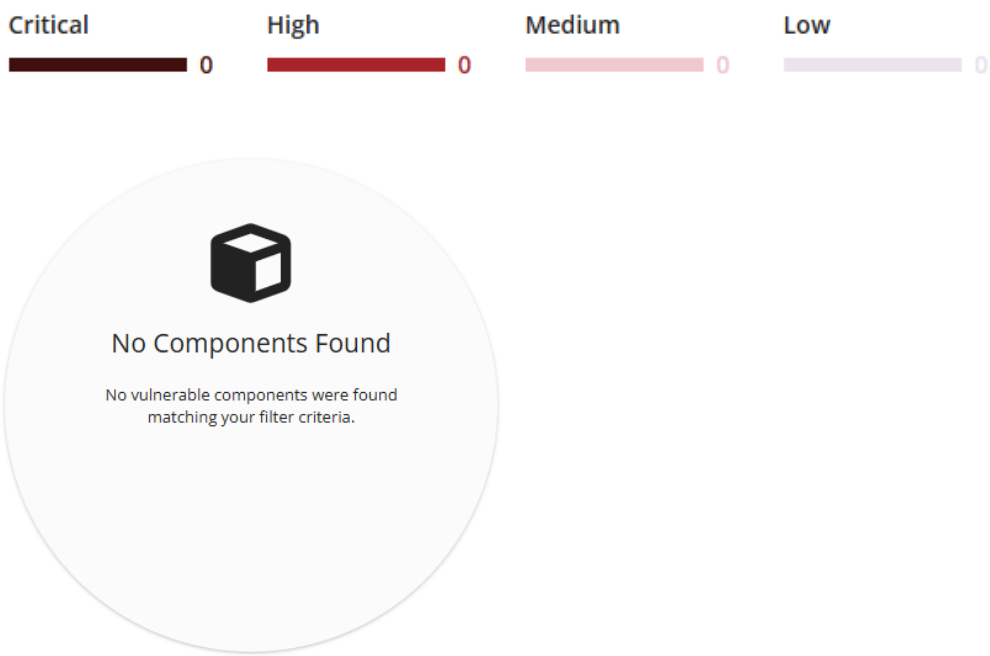
79 **Gitlab**

80 No vulnerabilities were identified for the terms regarding with the TOE.

81 **Black Duck scanning**

82 The evaluator has performed the source code scan using the Synopsys Black Duck tool to find vulnerable modules that are used in the source code.

83 After performing the analysis, the Black Duck tool did not find any vulnerable components in the source code.

84 

85

7 Code analysis

7.1 Motivation

86 This section refers to the static and manual code analysis performed by the evaluator based on the source code provided by the manufacturer.

87 Understanding the inner workings of the source code is very important. Static source code analysis offers a powerful method to dissect the module's architecture and identify potential vulnerabilities.

88 Static code analysis combined with manual verification allows the evaluator to check the quality of the code and locate possible implementation errors.

7.2 Test purpose

62 Assess source code quality and search for potential vulnerabilities in the source code.

7.3 Tested sample

63 The code packages shared by the manufacturer are as follows:

File name	Description	Hash (Algorithm: MD5)
aw693w-a1-bootrom-src.tar	ROM source code	54DC009747B72C1F08AC5C75641D2BD2
AW693_security_related_source_code.zip	Security source code	62D70547A974B754AD5E3EA5084A2243

7.4 Description

89 This test provides an analysis of the identified issues in the static code analysis, categorized by severity levels and issue types. The objective of the analysis has been the source code installed in the internal memory of the TOE AW692, including the bootloader and security related source code.

64 Some of the tools used for static analysis are as follows:

- Coverity (Synopsys): An advanced static analysis tool that detects defects in C, C++, Java, and other programming languages. It provides

early detection of security vulnerabilities and coding issues. (Website: https://www.synopsys.com/software-integrity/security-testing/static-analysis-sast.html)

- Cppcheck: Analyzes C and C++ code without requiring compilation, detecting common errors and security issues (Website: https://cppcheck.sourceforge.io/).
- Flawfinder: Focuses on code security, scanning files for potentially vulnerable patterns in C and C++. It only requires source files. (Website: https://dwheeler.com/flawfinder/).
- Splint: Statically analyses C code to detect logical errors and enforce good coding practices, without requiring compilation (Website: http://www.splint.org/).
- CppDepend: A static analysis and code quality tool for C and C++. It provides metrics, dependency visualization, architecture validation, and technical debt estimation (Website: https://www.cppdepend.com/).

65 The possible errors encountered during the analysis have been grouped according to their level of severity. In addition, within each severity level, different categories of errors can be found, where several of them can be preliminarily discarded as "false positives" or "bad practices" in the code style.

66 This classification helps in prioritizing actual errors while avoiding unnecessary fixes for false positives, ensuring a more efficient debugging process.

7.4.1 Security Levels:

- Information: This category includes all those errors that can be interpreted as false positives, such as the reference to the use of external libraries that are not included in the project to be analysed. Nevertheless, the developer needs to pay attention to them and verify to rule out possible real errors.
- Style: At this level, "bad practices" of coding are cited. While these advisories have no direct impact on security, unoptimized code integration can trigger unwanted and unpredictable states during execution.
- Warning: In this category you can find cases similar to those found in the categories of "information" and "style". However, in this category are highlighted those sections of code that can compromise potential pieces of code during execution, so it would be necessary to pay attention to them and review their usage.

- Error: The cases included in this category require immediate attention as they can cause serious runtime errors. However, preliminarily some of these cases can be classified as false positives due to missing external files, third-party libraries or compiler parameters, which after being checked by the developer can be completely discarded.

7.4.2 Issue Categorization:

- **Information Level Issues:**
  - missingInclude and missingIncludeSystem: These cases can be classified as false positives, as they are due to problems in accessing external library files from the manufacturer. However, it is necessary to pay attention to them and verify that the libraries that are declared are included in the final project.
  - Other issues: All other cases under the "information" category are also considered false positives, as they correspond to multiple traces used for code analysis. In this category, the following issues can be found:
    - normalCheckLevelMaxBranches: The tool exceeded the number of traces applied to specific file with the current configuration.
    - noValidConfiguration: Files that are incompatible with the current configuration/tool.
    - toomanyconfigs: The tool skips the analysis of code section containing multiple redundant configurations.
- **Style Level Issues**
  - unusedVariable, unusedFunction, unusedStructMember and unreadVariable: These cases indicate variables, functions or other structs declared but never used. If they are intentional placeholders for future implementation, they can be ignored. Otherwise, they should be removed or utilized properly.
  - redundantInitialization, redundantAssignment, redundantAssignInSwitch: Redundancy in the initialization of variables, assignment of values without having used the previously defined values and duplicity in the assignment.
  - duplicateExpression: Expressions duplicated in both directions of a comparison or similar.
  - badBitmaskCheck: Redundancy in the use of logical operators when expected to get null values.

- constParameterPointer and constVariablePointer: Variables and parameters that could be declared as pointers for optimal code execution.
- knownConditionTrueFalse: The use of redundant conditions, comparisons, or logical operations where the result will always be the same.
- shadowVariable: A possible misuse of local variables with the same name as other external, global, or similar variables.
- variableScope: The scope of a variable is greater than actually used and could be reduced to take up less memory space, avoid redundancies or misuse that conflicts with other variables and thus have a more optimal code execution.

- **Warning Level Issues**
  - identicalInnerCondition: Similar than knownConditionTrueFalse, where the use of redundant conditions, comparisons, or logical operations, where the result will always be the same.
  - identicalConditionAfterEarlyExit: A return value identical to the condition that gave access, so the developer need to pay attention to the logic used in that section of code.
  - constStatement: Code redundancy for unused sections of code.

- **Error Level Issues**
  - syntaxError: Errors classified as syntax errors should be carefully reviewed. If they are related to missing semicolons, unmatched brackets, or other fundamental mistakes, they need fixing. However, if they are due to preprocessing directives or dependencies, they could be false positives.
  - unknownMacro and preprocessorErrorDirective: Preliminary, these cases can be classified as false positives because they are due to missing configuration from external files/libraries, similar to the "missingIncludeSystem" case in the “Information” category. Nevertheless, it is mandatory to pay attention to them because a partial or total problem of any of these parameters can lead to runtime crashes.
  - uninitvar: These errors correspond to previously uninitialized variables, and need to be verified first-hand by the developer. In most cases, these errors are linked to missing external libraries, information or configuration, and can be considered in part as false positives, but they need to be verified by the developer.

- implicitConversion: Implicit type conversions should be reviewed, as they may lead to data loss or unexpected behavior. It is recommended to use explicit casting where necessary.
- possibleNullPointer: These cases require special attention, as null pointer dereferencing can lead to runtime crashes. Proper checks should be added before pointer usage.

7.5 Results

67 The evaluator has performed the analysis using the Coverity, Cppcheck, Flawfinder, Splint and CppDepend tools and have grouped all the results found in the files [NXP_IW692-AVA_BootROM_Annex_v1.0] and [NXP_IW692-AVA_Security_src_Annex_v1.0].

68 The general results obtained through the static analysis performed are shown below:

7.5.1 BootROM code module results

90 The overall results of the static analysis of the BootROM code are as follows:

91

Severity	Case counter	%
error	177	4.50
information	3431	87.33
style	318	8.09
warning	3	0.08

92 And the cases found in each of the categories specified above are the following:

93

Severity	Category	Case counter
error	preprocessorErrorDirective	1
error	syntaxError	4
error	uninitvar	1
error	unknownMacro	171
information	missingInclude	1365
information	missingIncludeSystem	2066
style	badBitmaskCheck	4
style	constParameterPointer	6
style	constVariablePointer	17
style	knownConditionTrueFalse	3
style	redundantAssignment	5
style	unreadVariable	174
style	unusedFunction	77
style	unusedStructMember	1
style	unusedVariable	15
style	variableScope	6
style	useStandardLibrary	4
style	shadowVariable	2
style	duplicateExpression	4
warning	identicalInnerCondition	2
warning	constStatement	1

7.5.2 Security related source code module results

94 The overall results of the static analysis of the security related source code are as follows:

95

Severity	Case counter	%
error	68	2.80
information	793	32.69
style	1538	63.40
warning	27	1.11

96 And the cases found in each of the categories specified above are the following:

Severity	Category	Case counter
error	preprocessorErrorDirective	3
error	syntaxError	41
error	unknownMacro	6
error	legacyUninitvar	1
error	resourceLeak	6
error	uninitStructMember	10
error	ctuOneDefinitionRuleViolation	1
information	missingInclude	423
information	missingIncludeSystem	370
style	badBitmaskCheck	3
style	constParameterPointer	203
style	constVariablePointer	350
style	knownConditionTrueFalse	58
style	redundantAssignment	27
style	redundantInitialization	6
style	unreadVariable	211
style	unusedFunction	282
style	unusedStructMember	7
style	unusedVariable	24
style	variableScope	133
style	duplicateAssignExpression	1
style	constParameter	8
style	selfAssignment	157
style	constParameterCallback	38
style	unsignedLessThanZero	1
style	unassignedVariable	1
style	arrayIndexThenCheck	1
style	unusedLabelConfiguration	2
style	redundantCondition	3
style	shadowVariable	3
style	duplicateCondition	2
style	constVariable	7
style	cstyleCast	6
style	duplicateExpression	4
warning	identicalInnerCondition	1
warning	nullPointerRedundantCheck	9
warning	funcArgOrderDifferent	1
warning	uninitMemberVar	16

97

7.5.3 Minor warnings: “Information” and “style” severity

98 In both code modules, the majority of the case detected fall under the "information" and “style” severity level.

- **“Information” severity cases**

74 In the case of "information" severity level, all notable cases are categorized as "missingInclude"/”missingIncludeSystem”. As mentioned, these cases are interpreted as false positives due to problems in accessing external libraries that are not included in the project. However, it is necessary for the developer to verify that they are included in the final construction of the project, but they are not critical since they are external dependencies that can be assumed to be correct. and whose correctness is beyond the possibility of the evaluator to verify.

75 As for the cases of "style", as mentioned above, they are largely associated with "best practices" during the code development process, and therefore, they are interpreted as recommendations for the developer to consider and improve code optimization. Specifically, proper code optimization should be applied for unused variables, functions and structs to improve code quality.

- **“Style” severity cases**

76 In the case of “Style” severity level, most cases are included in the following categories:

- Unused values in variables ("unreadVariable" category).
- Unused functions (“unusedFunction” category).

77 For unused values in variables in the BootROM code module, most cases come from directories that include what are possibly examples of function use. The main recommendation for these cases is to review and remove the example directories from the use of functions and only include the functional code with functions that are actually used in future code updates.

78 Some of the directories that include these code examples are located at: “\aw693w-a1-bootrom-src\common\clns-blackbird_cosim_PRC\examples”.

79 For all other cases, the evaluator interprets these cases as false positives due to external dependencies. The evaluator assumes that many of the functions, variables, and dependencies are included in external code modules that are not included in the scope of the evaluation. However, as in previous cases, the developer is advised to manually review these cases to rule them out completely.

80 The last notable cases within the "style" severity correspond to the category of “constParameterPointer” and “constVariablePointer”. As mentioned above, these two categories correspond to code optimization.

81 The "constParameterPointer" and “constVariablePointer” categories refers to variables that can be declared as pointers, optimizing the flow of code execution.

82 In addition, it is important to highlight the cases of the "variableScope" category in terms of code optimization, since this category refers to a variable that can reduce its scope, thus reducing the size of memory reserved during runtime.

83 Like all categories included within the "style" severity, the latter must be reviewed by the developer, verifying that these sections of code can actually be optimized and do not affect the intended code execution.

84 For more information, see the [NXP_IW692-AVA_BootROM_Annex_v1.0] and [NXP_IW692-AVA_Security_src_Annex_v1.0] files corresponding to the static code analysis results annex of the mentioned code modules.

7.5.4 “Warning” severity cases

85 For the warning issues, the few cases reported by static analysis correspond to code redundancies, and problems in the definition of some conditions. These should be reviewed by the developer, although it does not imply that they can trigger a malfunction of the code execution.

69 The main cases of code redundancy obtained during the analysis correspond to:

(A) “identicalInnerCondition” in the files:

- “\aw693w-a1-bootrom-src\common\src\cssv2.c”
- “\AW693_security_related_source_code\wifi_sec_if\Driver\mlan\mlan_misc.c”

(B) “nullPointerRedundantCheck” in the files:

- “\AW693_security_related_source_code\wifi_sec_if\Driver\mlan\mlan_scan_offload.c”
- “\AW693_security_related_source_code\wifi_sec_if\Driver\moal\linux\moal_sdio.c”

86 The evaluator recommends that the developer pay special attention to these warning cases, as they are localized in specific source code files. However, these cases can be interpreted as false positives, since part of the integrity of the code can be supported by code redundancy, in order to prevent a state variable or condition from causing an unexpected error during execution.

87 On the other hand, an error related to uninitialized variables has been found. The same error is found in the files:

- \AW693_security_related_source_code\wifi_sec_if\Driver\nanotls\tests\src\nanotls-test-data-fuzz.cpp
- \AW693_security_related_source_code\wifi_sec_if\Driver\nanotls\tests\src\nanotls-test-data-negative.cpp
- \AW693_security_related_source_code\wifi_sec_if\FW\nanotls\tests\src\nanotls-test-data-fuzz.cpp
- \AW693_security_related_source_code\wifi_sec_if\FW\nanotls\tests\src\nanotls-test-data-negative.cpp

88 Below is a screenshot of the mentioned uninitialized variables case:

89
```
NegativeDataTest()
{
    randomFill(&host_ctx, sizeof(host_ctx));
    randomFill(&device_ctx, sizeof(device_ctx));
    randomFill(&host_hello, sizeof(host_hello));
    randomFill(&device_hello, sizeof(device_hello));
    randomFill(&host_finished, sizeof(host_finished));
    randomFill(&host_keys, sizeof(host_keys));
    randomFill(&device_keys, sizeof(device_keys));

    EXPECT_EQ(nanotls_host_init(&host_ctx, ecdsa_keypair.pub), E_OK);
    EXPECT_EQ(nanotls_device_init(&device_ctx, ecdsa_keypair.priv, NULL), E_OK);
}
```

90 This case is included within the "\AW693_security_related_source_code\wifi_sec_if\Driver\nanotls\tests\" directory, which appears to be the developer test directory. Therefore, the evaluator considers the latter case as a false positive, since the content of this directory does not affect the expected functioning of the TOE.

91 In addition, as in the previous cases with errors in the example directories, the main recommendation for these cases is to review and remove the test directories from the use of functions and only include the functional code with functions that are actually used in future code updates.

7.5.5 “Error” severity cases

70 As mentioned in the previous case, the errors classified under "unknownMacro", “preprocessorErrorDirective” and “uninitvar” or similar are classified as “false positives” due to missing external library, source files or configuration files from the developer build environment, similar to the "missingInclude" and other cases included in the “information” severity level. However, it is recommended that the developer pays special attention to these messages and verifies that everything is defined according to their criteria.

71 Regarding the errors of the category “syntaxError”, they need manual verification by developer. Preliminary, it is difficult for them to be "real"

positives, since, for example, they would not allow the compilation of the project.

72 However, there are several syntax errors that need to be addressed below.

73 The first error of the "syntaxError" category is found in the BootROM code module. The evaluator reviewed the file "\aw693w-a1-bootrom-src\common\seboot\src\seboot_hal.c". The following screenshot shows this "syntaxError" located on line 344:

92
```
340 static inline seboot_status_t seboot_hal_get_puf_config(seboot_puf_config_t *pufConfig)
341 {
342     u64 *OTP_sq_offset = (u64 *)OTP_BUFFER;
343     /* No need to check the input arguments for this inline functions. */
344     assert(pufConfig);
```

93 The error of the "syntaxError" category of the "assert()" function can be interpreted as a possible false positive since there are no syntax errors in the line of the previous screenshot. In addition, the use and definition of the function in other parts of the BootROM code is error-free, as can be seen in the following screenshot extracted from the same file with the function definition:

94
```
52 #define assert(x)
53 #endif
```

95 Therefore, the evaluator determines that the error "syntaxError" in the function "assert()" is a false positive. In addition, the false positive may come from a problem with the variable "pufConfig" and should be reviewed by the developer, although this variable does not throw any kind of error in the static analysis results.

96 The second "syntaxError" error is found on line 469 of the "\aw693w-a1-bootrom-src\common\src\nv.c" del BootROM code module:

97
```
454 #if SEBOOT==1
455         DIF(0, CIU.CIU_CPU1_CPU2_FW_DWLD_CTRL2.BF.LCS_READY, HARD_EQ, 100,
456             {
457                 NXP_CSSL_FP_FUNCTION_CALL_BEGIN(lcs_result, token, seboot_process_lcs());
458                 if ((NXP_CSSL_FP_FUNCTION_CALLED(seboot_process_lcs) != token) || (kStatus_SEBOOT_Success != lcs_result))
459                 {
460                     dvprintf("CPU%d: token %x expected token %x result %x expected result %x",
461                             CPU,token, NXP_CSSL_FP_FUNCTION_CALLED(seboot_process_lcs),
462                             lcs_result, kStatus_SEBOOT_Success);
463                     SE_ERROR(SE_SEC_ERROR);
464                 }
465                 NXP_CSSL_FP_FUNCTION_CALL_END();
466 #else
467                 process_lcs();
468 #endif
469             });
```

98 This error is due to the use of the "SEBOOT==1" condition, and the closing of "DIF()" after the else() condition, which static analysis tools interpret as a false "unmatched" error considering that the "});" closure of line 469 is isolated. Therefore, the evaluator discards this error and classifies it as a false positive.

99 The same cases are found in the security related source code module in the following files:

- "\AW693_security_related_source_code\wifi_sec_if\Driver\mlan\mlan_join.c (**line 1183**):

```
1183 {
1184     mlan_status ret = MLAN_STATUS_SUCCESS;
1185     mlan_adapter *pmadapter = pmpriv->adapter;
1186     HostCmd_DS_802_11_ASSOCIATE *passo = &cmd->params.associate;
1187     BSSDescriptor_t *pbss_desc;
1188     MrvlIEtypes_SsIdParamSet_t *pssid_tlv;
1189     MrvlIEtypes_PhyParamSet_t *pphy_tlv;
1190     MrvlIEtypes_SsParamSet_t *pss_tlv;
1191     MrvlIEtypes_RatesParamSet_t *prates_tlv;
1192     MrvlIEtypes_AuthType_t *pauth_tlv = MNULL;
1193     MrvlIEtypes_RsnParamSet_t *prsn_ie_tlv = MNULL;
1194 #ifdef WPA3_SAE
1195     MrvlIEtypes_SAE_PWE_Mode_t *prsnx_ie_tlv = MNULL;
1196 #endif /* WPA3_SAE */
1197     MrvlIEtypes_SecurityCfg_t *psecurity_cfg_ie = MNULL;
1198     MrvlIEtypes_ChanListParamSet_t *pchan_tlv;
1199     WLAN_802_11_RATES rates;
1200     t_u32 rates_size;
1201     t_u16 tmp_cap;
1202     t_u8 *pos;
1203     IEEEtypes_CapInfo_t *pcap_info;
```

- \AW693_security_related_source_code\wifi_sec_if\Driver\mlan\esa\AssocAp_srv_rom.h (**line: 135**):

```
132 SINT32 assocSrvAp_CheckSecurity(cm_Connection *connPtr,
133                                 IEEEtypes_WPSElement_t *pWps,
134                                 IEEEtypes_RSNElement_t* pRsn,
135                                 IEEEtypes_WPAElement_t *pWpa,
136 #ifdef WAPI_AP
137                                 IEEEtypes_WAPIElement_t *pWapi,
138                                 IEEEtypes_StatusCode_t  *pResult
139 #endif
140                                 );
```

100 Therefore, as in the previous "unmatched" case of BootROM, the evaluator determines that all these cases found within the security related source code are considered false positives.

101 On the other hand, the static analysis tools have found several errors related to the use of "ENTER();" in different files of the security related source code. Below are the screenshots of these cases:

- \AW693_security_related_source_code\wifi_sec_if\Driver\mlan\esa\authenticator_api.c: (**line 839**):

```
833 {
834     hostsa_util_fns *putil_fns = (hostsa_util_fns *)psa_util_fns;
835     hostsa_mlan_fns *pmlan_fns = (hostsa_mlan_fns *)psa_mlan_fns;
836     mlan_status ret;
837     hostsa_private *priv = MNULL;
838
839     ENTER();
```

- \AW693_security_related_source_code\wifi_sec_if\Driver\mlan\esa\keyMgmtAp.c (**line 196**):

-
```
189 void KeyMgmtSendGrpKeyMsgToAllSta(hostsa_private *priv)
190 {
191     hostsa_mlan_fns *pm_fns = &priv->mlan_fns;
192     apKeyMgmtInfoSta_t *pKeyMgmtInfo;
193     cm_Connection *connPtr =MNULL;
194     t_void *sta_node =MNULL;
195
196     ENTER();
```

- \AW693_security_related_source_code\wifi_sec_if\Driver\mlan\esa\keyMgmtApStaCommon.c (**line 167**):

-
```
161 static t_u32 srand_new(void  *priv)
162 {
163     phostsa_private psapriv = (phostsa_private) priv;
164     hostsa_util_fns  *util_fns = &psapriv->util_fns;
165     t_u32 sec, usec;
166
167     ENTER();
168     get_system_time(util_fns, &sec, &usec);
169     sec  = (sec & 0xFFFF)  + (sec >> 16);
170     usec = (usec & 0xFFFF) + (usec >> 16);
171
172     LEAVE();
173     return (usec << 16) | sec;
174 }
```

- \AW693_security_related_source_code\wifi_sec_if\Driver\mlan\mpl\mpl_main.c (**line 190**):

-
```
186 mlan_status mpl_tx_packet(t_void *pmpl_priv, mlan_buffer *pmbuf)
187 {
188     mlan_status ret = MLAN_STATUS_SUCCESS;
189     pmpl_private priv = (mpl_private*)pmpl_priv;
190     ENTER();
191
192     if (priv)
193         ret = mpl_got_tx_packet(priv, pmbuf);
194     LEAVE();
195     return ret;
```

- \AW693_security_related_source_code\wifi_sec_if\Driver\mlan\mpl\mpl_packet.c (**line 201**):

-
```
197 static t_void mpl_fill_snap_hdr(pmpl_private priv, mpl_snap_header * snap_hdr,
198                                 t_u8 num_tlvs)
199 {
200     pmpl_util_fns pmu = mpl_get_utils();
201     ENTER();
202
203     memset(pmu, snap_hdr, 0x00, sizeof(mpl_snap_header));
204     memcpy_ext(pmu, snap_hdr->llc, mpl_llc, sizeof(mpl_llc),sizeof(snap_hdr->llc));
205     memcpy_ext(pmu, snap_hdr->oui, mpl_oui, sizeof(mpl_oui),sizeof(snap_hdr->oui));
206     snap_hdr->type = MPL_SNAP_HEADER_TYPE;
207     snap_hdr->version = MPL_SNAP_HEADER_VERSION;
208     snap_hdr->min_extensions = priv->sched.aw_start_info.min_ext;
209     snap_hdr->presence_mode = MPL_PRESENCE_MODE_AVAILABLE_EACH_WINDOW;
210     snap_hdr->sequence_number = wlan_cpu_to_le16(mpl_get_tx_seq_num());
211     snap_hdr->flags.num_tlvs = num_tlvs;
212
213     LEAVE();
214 }
```

- \AW693_security_related_source_code\wifi_sec_if\Driver\mlan\mpl\mpl_peer_db.c (**line 147**):

-
```
143 static t_void mpl_peer_clean(pmpl_private priv, mpl_peer_info *peer,
144                                 t_bool rem_assoc_queues)
145 {
146     pmpl_util_fns  pmu = mpl_get_utils();
147     ENTER();
148
149     if (peer->sync_params) {
150         free(pmu, (t_u8*)peer->sync_params);
151         peer->sync_params = MNULL;
152     }
153     if (peer->my_queues && rem_assoc_queues) {
154         peer->my_queues->peer = MNULL;
155         util_unlink_list(pmu->pmoal_handle, &priv->queues.q_peers,
156                          (pmlan_linked_list)peer->my_queues, MNULL, MNULL);
157         mpl_queue_clean_one_peer(priv, peer->my_queues);
158         peer->my_queues = MNULL;
159     }
160
161     PRINTM(MDAT_D, "%s: removed peer %p mac="MACSTR"\n",
162            __FUNCTION__, peer, MAC2STR(peer->mac_addr));
163     free(pmu, (t_u8*)peer);
164     priv->peer_db.num_peers--;
165     LEAVE();
166 }
```

102 And the same errors are found in the case of "LEAVE();" in the following two cases:

- \AW693_security_related_source_code\wifi_sec_if\Driver\mlan\mpl\mpl_queue.c (**line 142**):

- ```
  137 static t_void mpl_queue_clean_pkts(pmpl_private priv, pmlan_list_head head)
  138 {
  139     pmpl_util_fns  pmu = mpl_get_utils();
  140     pmpl_mlan_fns  pml = &priv->mlan_fns;
  141     pmlan_buffer   pmbuf;
  142     ENTER();
  143
  144     while ((pmbuf = (pmlan_buffer) util_peek_list(pmu->pmoal_handle,
  145                                                   head, MNULL, MNULL))) {
  146         util_unlink_list(pmu->pmoal_handle, head,
  147                          (pmlan_linked_list)pmbuf, MNULL, MNULL);
  148         /* return packet as failed for cleanup */
  149         pmbuf->buf_type = MLAN_BUF_TYPE_DATA;  /* need set for below call */
  150         pml->mlan_write_data_complete(pml->pmlan_adapter,
  151                                       pmbuf, MLAN_STATUS_FAILURE);
  152     }
  153     LEAVE();
  154 }
  ```
- \AW693_security_related_source_code\wifi_sec_if\Driver\mlan\mpl\mpl_sched.c (**line 157**):
- ```
  136 static mlan_status mpl_sched_new_peer_dat(pmpl_private priv, mpl_peer_info *peer)
  137 {
  138     mlan_status ret = MLAN_STATUS_FAILURE;
  139     pmpl_util_fns pmu = mpl_get_utils();
  140     mpl_sched_peer_data *sch_peer;
  141
  142     ret = malloc(pmu, sizeof(mpl_sched_peer_data), (t_u8**)&sch_peer);
  143     if ((ret != MLAN_STATUS_SUCCESS) || (!sch_peer)) {
  144         PRINTM(MERROR, "%s: cannot allocate sch_peer.\n", __FUNCTION__);
  145         goto done;
  146     }
  147     memset(pmu, sch_peer, 0x00, sizeof(mpl_sched_peer_data));
  148     sch_peer->peer = peer;
  149     peer->my_sched_pdat = sch_peer;
  150     util_enqueue_list_tail(pmu->pmoal_handle,
  151                            &priv->sched.sched_peers,
  152                            (pmlan_linked_list)sch_peer,
  153                            MNULL, MNULL);
  154     priv->sched.num_peers_in_sched++;
  155
  156 done:
  157     LEAVE();
  158     return ret;
  159 }
  ```

99 After review, the evaluator considers these errors to be "false positives" as they are similar to "undefined variables" which are common when isolating code modules during static analysis and the tools do not find the definition or configuration of these functions because they are in external code modules external to those included in the analysis.

100 It is expected that these functions have been correctly defined by the developer in other code modules and/or libraries that are not included within the scope of this assessment.

101 Finally, the static analysis tools have found a different "syntaxError" case in the following file:

- \AW693_security_related_source_code\bt_sec_if\Driver\btnxpuart.c (**line: 2270**):

- ```
  2270 static const struct of_device_id nxpuart_of_match_table[] __maybe_unused = {
  2271     { .compatible = "nxp,88w8987-bt", .data = &w8987_data },
  2272     { .compatible = "nxp,88w8997-bt", .data = &w8997_data },
  2273     { }
  2274 };
  ```

103 However, the evaluator does not see any syntax errors in the previous screenshot, so this error is considered as a false positive that may be related to the definition and use of "__maybe_unused". Therefore, the evaluator recommends to the developer the manual review of this case, although he considers that it is not a failure that can trigger a real failure in the operation of the TOE.

104 In conclusion, for the code module of the bootloader and security related features, the evaluator determines that the cases reported as "errors" are interpreted as "false positives", in most cases due to the access to external files and environment configuration.

105 But it is up to the developer to review these cases for future code reviews to be totally discarded, since it should not be forgotten that these cases could cause serious runtime errors. The rest of the results obtained correspond mostly to "bad practices" that must be reviewed by the developer to optimize the execution flow.
  ```

8 Vulnerability analysis

102 This section presents a vulnerability analysis of the TOE, aiming to identify technical weaknesses, assess their potential impact.

103 The objectives of the analysis are:

- Identify vulnerabilities in the current system design and implementation.
- Assess the potential impact of these vulnerabilities under realistic attack scenarios.
- Verify the consistency between implemented security functions and the security objectives stated in the official documentation.
- Support the technical team in making informed decisions regarding security improvements.

104 The analysis focuses on critical areas of the system, including:

- Firmware security properties related to secure update and secure initialization.
- Secure communication functionality.
- Life cycle states and access to sensitive functionalities (e.g., secure debug). Countermeasures for detection and protection against logical and physical attacks.

105 The evaluation was carried out through the following steps:

- In-depth review of the Security architecture manual, assessing the security properties and how they are implemented.
- Incorporation of existing design analysis and prior comments, focusing on known concerns and previous findings.
- Comparison with the Security Target, ensuring that the implementation aligns with the stated conformance rationales.
- Cross-checking with additional documentation, seeking inconsistencies or potential new vulnerabilities.
- Identification of high-level attack scenarios for each functional area of the system.

- Selection and prioritization of key properties for penetration testing, based on risk and exposure.
- Definition, execution, and documentation of penetration tests, with evidence gathered to support each finding.

106 The areas have been defined based on the security functional requirements included in [ST_10].

SFRs	Area
Verification of Platform Identity	Identity
Verification of Platform Instance Identity	Identity
Secure Initialization of Platform	Secure Initialization
Secure Update of Platform	Firmware Update
Physical Attacker Resistance	Countermeasures and Physical attacks
Secure Communication Support – RCP-Host	Secure Communication
Secure Communication Support – RCP-Network	Secure Communication
Secure Debugging	System LifeCycle and Debug Authentication
Availability support	Countermeasures and Physical attacks

8.1 Identity

107 The platform stores in OTP fuses a 128-bit IETF RFC4122 compliant non-sequential Universally Unique Identifier (UUID). It can be read through the bring-up process for Wi-Fi interface via USB host interface.

108 The unique identification of the platform can also be checked physically on the package.

8.1.1 Input to Penetration Testing

109 In the case of the identity of the device, the evaluator determines that there is no relevant attack scenario to be studied, since the identity is stored in the OTP memory, which is immutable.

8.2 Secure Initialization

110 Extracted from the [AW/IW692_SAM]:

111 “Secure boot ensures authenticity, integrity and confidentiality of the device bootloader, firmware, and other software during the boot process and ensures that the intended secure life-cycle state is reached. ECDSA P-256 with SHA-256 are there to guarantee authenticity and integrity of the firmware image. AES-based encryption is used to ensure code confidentiality while the firmware image is stored outside the device. Boot ROM together with the ELS is the immutable RoT in charge of enforcing secure boot and it does so according to the policies defined by the life-cycle state”.

112 In relation to the boot flow, initially at power on reset there are two CPUs and both of them are taken out of reset and both of them will begin execution of their own boot ROMs. “It should be considered that CPU1 (the WLAN CPU) is to the master during this boot operation and CPU2, the narrow band CPU is in slave mode.” The boot flow is described in detail in the NXP Security Reference Manual [AW/IW692_SAM] and outlined and analysed in the continuing.

113 In addition to the two CPUs, the OTP memory, the S50 subsystem and the physically unclonable function (PUF) are involved in the boot process.

114 The main boot flow could be summarized in the following stages:

1. BIST Wait (Buit-In Selft-Test)
2. MPU Init
3. S50 Init
4. Reading of the OTP
5. Hardening, GDET init
6. Host init.
7. Jump to firmware

115 The BIST (Buit-In Selft-Test) wait phase is important because the boot of a TOE depends on critical hardware blocks being functional before any software is executed. Extracted from the [AW/IW692_SAM]: “In the BIST Wait phase, both cores begin to poll a bit from the ATE team that indicates when the BIST logic has finished its repair installation. There are redundant ram columns built into the device to increase yield, if during test, a single bit failure in a big ram is detected, rather than tossing the part, a circuit

controlled by on-chip non-volatile storage can swap out a bitline and replace an entire column. This is recorded in these on-part notations, the processing of those notations and the resultant substitution must be completed before the bootcode can use the cells. This processing period is denoted by the time the BIST circuit ready bit remains low, once the period ends, or if a bootcode timeout elapses, the boot will proceed, ready or not".

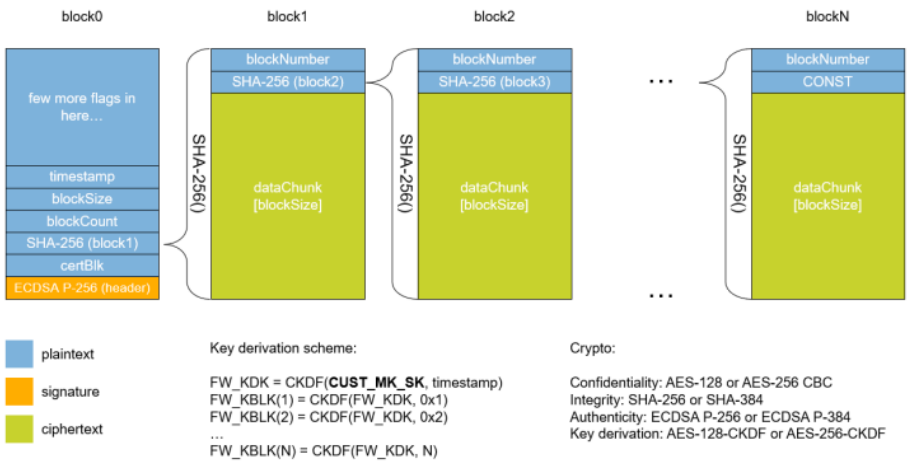
116 After that, both CPUs will initialise. These CPUs have the ARM memory protection IP and both BootROMs enable it in keeping with good security practices.

117 The rest of the boot sequence continues with the initialization of the hardening and glitch detector systems, which support the different security aspects of the TOE. Aspects such as the glitch detector will be covered in the section corresponding to the countermeasures of the TOE.

118 Another step to highlight is the reading of the OTP, which is crucial to e.g. ensure that the correct life-cycle state information is used. The OTP is read in a very specific way. The security lines of the OTP are read in a random order, not once, but twice, and then the results are compared to determine if they are consistent. This way, the timing of each OTP read is unpredictable, and, additionally, an attacker trying to affect the process through e.g. fault injection, will also have to repeat the effect over at least two points in time. The randomization uses a random number generator for the purpose.

119 The firmware verification process is based on the SB3.1 FW image format and the usage of SHA-256 for hashing and ECDSA P-256 for the signature providing authentication and integrity of the firmware.

120 Public keys are stored in certificate blocks and validated against a hash of hashes in OTP, with additional revocation bits checked. Private keys are assumed to be securely managed and are out of scope. The certificate block and the SB3.1 image format can be seen in the figure below.

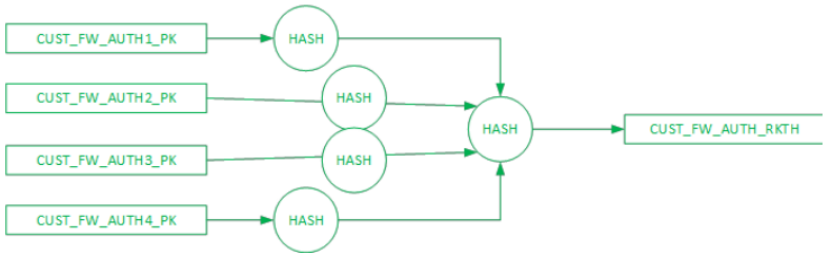


121

122 Key properties of the image format are:

- The hash for the ECDSA signature includes (recursively) all fields of the image. The hash of the previous block of the firmware image is included in the next image block, creating a chain, where the last hash, in block0, is created based on recursive hash values. Furthermore, hash value in block0 covers all fields in block0 as well, and is the hash that is signed.

- The private firmware signature key is used for signing the hash. The corresponding public key, together with optionally more public keys, are part of the certificate block. In the signature verification process, certificate clock is hashed and compared to the immutable hash value stored in OTP from the provisioning (see figure below). This procedure provides the assurance that the signing key is corresponding to an authenticated, trusted public key, together with the verification of the revocation bits in OTP.



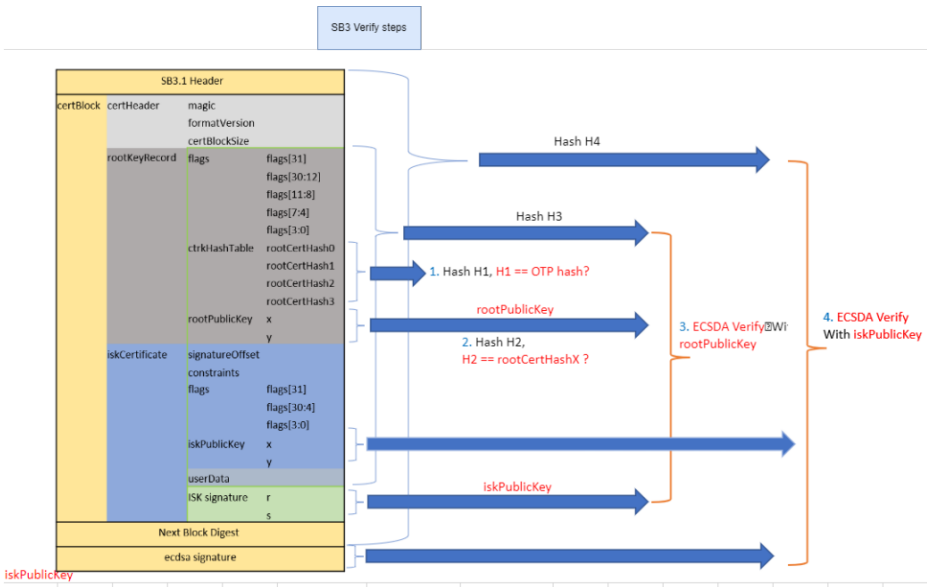
- The version number is included in the signed header, and this value is compared to the version number in OTP, to mitigate downgrade attacks. This prevents older, potentially vulnerable firmware from being executed and the inclusion of the version number can be seen in more detail in the figure below, where the SB3 header format is illustrated.

- The SB3.1 format is assessed as secure and robust. For example, image extension attacks are infeasible, since the hash will be changed. Any modification in any of the blocks will propagate to the hash, that is signed. All fields are included in the hash.

SB3 Header Format

sb3 header	blockCount	uint32_t	Number of block not including the header block
	blockSize	uint32_t	Size in bytes of data block (repeated blockCount times for SB3 data stream)
	timestamp	uint64_t	
	firmwareVersion	uint32_t	Version number of the included firmware
	imageTotalLength	uint32_t	total image length in bytes, including signatures etc.
	imageType	uint32_t	image type and flags
	certificateBlockOffset	uint32_t	Offset from start of header block to the certificate block. This allows the signed image verification code to verify the signature over the header block
	description	uint8_t[16]	description
next blk	SHA-256 of block1 to use for block1 authentication and integrity	32bytes	Hash is computed over whole block1 after data chunk encryption.
certificate blk	certBlk	208/348bytes	If isk is not used 208 bytes, otherwise 348 byes
ecdsa signature	ecdsaSig	64bytes	the signature covers all the (header + next blk digest + certificate blk) data words
	Total size	364/504 bytes	

123 In order to perform the signature verification, a series of initial verifications of the SB3.1 image are performed based on hash comparisons, which is illustrated in the figure below:



124 Encryption is used of the image based on a scheme where a different encryption key is derived for each block. The confidentiality of the images is outside the scope of the evaluation, which means that further analysis of the implementation and properties of the confidentiality protection is not considered in this evaluation.

125 Finally, once all the sequences and boot checks have been successfully completed, ensuring the authenticity and integrity of the firmware installed

on the TOE, the jump to the firmware is executed, starting the expected normal mode of operation.

126 Security features to be considered during the boot flow process:

(A) **Randomization**: During the boot flow, reads of important fields use special anti-glitch code that will read a field, have a random delay and then re-read the field. There are many of these constructs as the boot progresses. This means that there is not an easy attack surface presented to an attacker to time a glitch since each boot will have a different timing before a field is consulted.

(B) **PLL**: The boot code enables the PLL in the SoC after OTP read. This permits patches to come in and change clock speeds or indeed the PLL enablement code itself.

(C) **Glitch Detector**: This mechanism is included in the S50 security hardware. The information and analysis are included in the section related to the countermeasures.

(D) **OTP locking**: The OTP can be locked for read or write, and write locking has two levels. The first level is in the OTP fuses itself, and the second level is the OTP memory controller. This controller has bits that can be used to lock reads and writes. These controller bits can be set during various LCS to open or close lines in the OTP. For example, the LCS can be written when the LCS is DD and JTAG is enabled. This allows for the LCS to be moved to FA. DD by itself w/o JTAG will write protect the LCS.

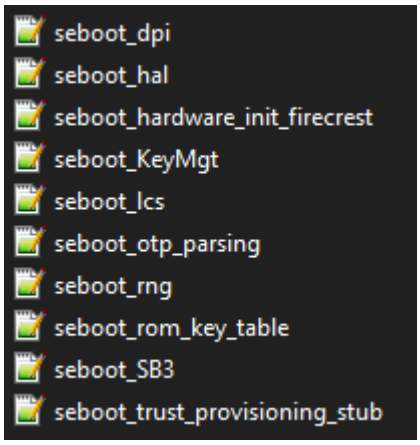
(E) **Public Key revocation**: There are 4 certificates available for signing the image. The certificate being used is noted in the SB3.1 header. There is the possibility that one or more of the certificates could have been compromised. If that were true, rogue images could be signed as if they were legitimate ones. To combat this during the life of the part and its releases, there is another OTP record that allows for this revocation.

8.2.1 Code implementation

127 The bootloader code module is implemented in the ROM memory and has been provided by the manufacturer via the [aw693w-a1-bootrom-src.tar] file.

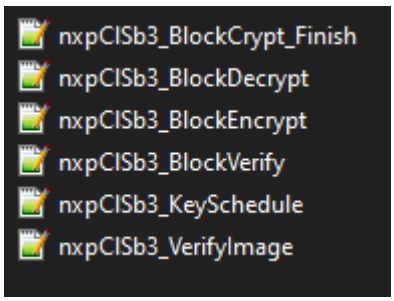
128 This code package includes parts such as the main boot routine for all cores, including the following:

(A) Secure boot routine: image verification, HAL, trust provisioning, key management and others.



(B) Cryptolib: Hash, PUF, memory and flow usage and protection.

(C) Verification of the SB3 image format and the different routines of: image and block verification, keys, decryption etc. within the folder: “\aw693w-a1-bootrom-src\common\seboot\nxpClSb3\src”



129 For the secure bootloader main features, the evaluator has verified that the main content is included in the folder: "\aw693w-a1-bootrom-src\common\seboot".

130 An example of a code implementation of the functions included in secure initialization is the verification of the headers of firmware images. Evidence of this can be found in the file "\aw693w-a1-bootrom-src\common\seboot\src\seboot_SB3.c", as in the following screenshot:

131

```
seboot_printf("verify header");
/* verify header */
if ( 0x33766273u != header->magic )          // string 'sbv3'
{
    seboot_printf("b)");
    NXP_CSSL_FP_FUNCTION_EXIT(seboot_sb3_load_manifest, kStatus_SEBOOT_Fail);
}
if ( 0x00030001u != header->formatVersion )  // version major = 3, minor = 1
{
    seboot_printf("c)");
    NXP_CSSL_FP_FUNCTION_EXIT(seboot_sb3_load_manifest, kStatus_SEBOOT_Fail);
}
if ( header->blockSize > NXPCL_SB3_MAX_BLOCK_SIZE_IN_BYTES )
{
    seboot_printf("d)");
    NXP_CSSL_FP_FUNCTION_EXIT(seboot_sb3_load_manifest, kStatus_SEBOOT_Fail);
}
if ( ( ( kSB3_TRUE == context->isUnencryptedFlag ) &&
       ( NXPCL_SB3_IMG_TYPE_SB3MANIFEST_PLAIN != header->imageType ) &&
       ( NXPCL_SB3_IMG_TYPE_SB3MANIFESTNXPKEYS_PLAIN != header->imageType ) ) ||
     ( ( kSB3_TRUE != context->isUnencryptedFlag ) &&
       ( NXPCL_SB3_IMG_TYPE_SB3MANIFEST != header->imageType ) &&
       ( NXPCL_SB3_IMG_TYPE_SB3MANIFESTNXPKEYS != header->imageType ) ) )
{
    seboot_printf("e)");
    NXP_CSSL_FP_FUNCTION_EXIT(seboot_sb3_load_manifest, kStatus_SEBOOT_Fail);
}
```

132 In the image above, the evaluator has checked different verification parts such as the magic number, encryption or image type. In case any of these checks is incorrect, the boot sequence status will be marked as "fail" via the "kStatus_SEBOOT_Fail" flag.

133 On the other hand, the evaluator has found evidence of good practices throughout the code. A clear example of this can be seen in the definition of the "True/false" states in the following figure related to SB3 format verification:

134

```
typedef enum _nxpClSb3_bool
{
    kSB3_TRUE                   = 0x3C5AC33Cu,    /*!< Value for TRUE.  */
    kSB3_TRUE256                = 0x3C5AC35Au,    /*!< Value for TRUE when P256 was used to sign the image.  */
    kSB3_FALSE                  = 0x5AA55AA5u,    /*!< Value for FALSE. */
    kSB3_OperationAllowed       = 0x3c5a33ccU,
    kSB3_OperationDisallowed    = 0x5aa5cc33U,
} nxpClSb3_bool_t;
```

135 With practices such as the one above, where states are defined from a more complex sequences than simple "0" or "1" values, a strong resistance is achieved against external attacks such as fault injection, which aim to bypass the verification conditions during the boot sequence.

136 In addition, in files such as "nxpClSb3_VerifyImage.c" included in the directory "\aw693w-a1-bootrom-src\common\seboot\nxpClSb3\src", the evaluator found different evidence of the verification of the certificates, as for example in the screenshot of the following image:

137

```
247     //check if root cert of ISK is used for signing
248     if (NXPCL_SB3_IMG_ROOTCERT_DIRECT_SIGN_TRUE != (rootCertFlags & NXPCL_SB3_IMG_ROOTCERT_DIRECT_SIGN_MASK))
249     {
250
251         if  (    (kSB3_RootKeyUsage_DebugCA_ImageCA_FwCA_ImageKey_FwKey != parms->soc_RoTNVM.soc_rootKeyUsage[rootCertIndex])
252                 &&  (kSB3_RootKeyUsage_ImageCA_FwCA != parms->soc_RoTNVM.soc_rootKeyUsage[rootCertIndex])
253                 &&  (kSB3_RootKeyUsage_DebugCA_ImageCA_FwCA != parms->soc_RoTNVM.soc_rootKeyUsage[rootCertIndex]))
254         {
255             NXP_CSSL_FP_FUNCTION_EXIT(nxpClSb3_img_authenticate_ecdsa, NXPCLSB3_STATUS_ERROR);
256         }
257
258         //calculate pointer to ISK certificate
259         uint32_t iskCertificateOffset = (offsetof(nxpClSb3_root_certificate_block_t, ctrkHashTable) + hashTableSize + rootSignatureSize)/sizeof(uint32_t);
260         if (iskCertificateOffset >= sizeof(nboot_sb3_block0_t))
261         {
262             NXP_CSSL_FP_FUNCTION_EXIT(nxpClSb3_img_authenticate_ecdsa, NXPCLSB3_STATUS_ERROR);
263         }
264         nxpClSb3_isk_block_t *pIskCertificate = (nxpClSb3_isk_block_t *)(((uint32_t *)pRootCert) + iskCertificateOffset);
265
266         //verify ISK certificate
267
268         //verify that ISK certificate hasn't been revoked
269         if (pIskCertificate->constraints < parms->soc_RoTNVM.soc_imageKeyRevocation)
270         {
271             NXP_CSSL_FP_FUNCTION_EXIT(nxpClSb3_img_authenticate_ecdsa, NXPCLSB3_STATUS_ERROR);
272         }
273
274         //verify signature of ISK block
275
276         if (pIskCertificate->signatureOffset >= sizeof(nboot_sb3_block0_t))
277         {
278             NXP_CSSL_FP_FUNCTION_EXIT(nxpClSb3_img_authenticate_ecdsa, NXPCLSB3_STATUS_ERROR);
279         }
280         //calculate length of rootkey and isk certificates
281         uint8_t *pIskSignature = (((uint8_t *)(pIskCertificate)) + pIskCertificate->signatureOffset);
282         uint32_t certLength = pIskCertificate->signatureOffset + iskCertificateOffset*sizeof(uint32_t);
283
284         //compute hash over certificates
285         NXP_CSSL_FP_FUNCTION_CALL(hashRetCode, nxpClHash_compute(session,
286                                                                  rootHashAlgo,
287                                                                  (uint8_t *)pRootCert,
288                                                                  certLength,
289                                                                  pWorkArea,
290                                                                  &hashOutputSize
291 ));
```

8.2.2 Input to Penetration Testing

138 There are several possible attack scenarios that are relevant to the secure initialization. Potential scenarios include:

- An attacker attempts to load and execute unauthorized code at boot, aiming to bypass the secure boot mechanism.
- An attacker attempts to exploit vulnerabilities in the input handling of boot data to compromise the integrity of the TOE.
- An attacker attempts to bypass secure boot by exploiting misconfigurations or improper enforcement of security settings, such as fuse states or lifecycle protections.
- An attacker tampers with firmware images, including headers, certificates, metadata, or signatures, to evade verification checks.
- An attacker targets weakness in signature verification, including cryptographic algorithm implementation, usage and firmware update format or error handling.
- An attacker attempts to compromise key handling of cryptographic verification mechanisms, aiming to misuse, modify or extract critical verification keys.
- For the above attack paths, the attacker can combine logical attacks together with fault injection, possibly affecting internal processing

and data, including modifying critical security values e.g. during read and processing, and the secure operation of verification mechanisms.

139 Based on these potential scenarios, some tests that could be carried out are:

- Test execution of unsigned binaries (raw or CRC-only images).
- Feed malformed certificate blocks, truncated manifests, unexpected lengths or duplicated fields.
- Attempt booting firmware signed with invalid or revoked certificates.
- Test that Secure Boot rejects any image not conforming to SB3.1 format or with corrupted manifests.
- Apply fault injection during critical steps of the secure initialization.

8.3 Firmware Update

140 The main security objective of the TOE's Secure Update mechanisms is to identify how the integrity and authenticity (confidentiality is not within scope) of the firmware is protected.

141 The secure update of AW692/AW693/IW623/IW693 follows the firmware download flow part of the secure boot, during which the firmware, and potentially patches, are verified using the same mechanisms.

142 Anti-rollback of firmware is ensured by the comparison of the version embedded with the firmware and a reference version stored in OTP. Only a firmware version greater or equal to the OTP reference version is accepted. This prevents older firmware versions from being reinstalled, protecting against potential known vulnerabilities in previous versions.

143 Integrity and authenticity are ensured through the ECDSA P-256 signature of a hash, which guarantees that the image has not been tampered with. The usage of ECDSA with the selected curves is assessed as best practice. Key revocation is handled by verifying against the values in the fuses, making it possible to transition securely to a new signing keys for secure update. Four certificates are available for signing the firmware. It is not allowed to revoke the key that is currently used.

144 Robustness against partial manipulation of blocks in the update package is achieved through a trust chain between the blocks: each data block includes the hash of the previous block, so that any partial modification of the payload can be detected in the final hash. The final hash, and the version number, is included in the signature, guaranteeing the integrity of the complete package and that the version number has not been modified. The comparison with the internally stored version number can then be securely performed on authenticated data.

8.3.1 Code implementation

145 Since the secure update mechanism follows the secure boot flow and the firmware images are verified using the same mechanisms, the evaluator has followed the same code revision as in the case of secure initialization.

146 Therefore, the evaluator considers that the verification of this mechanism is covered in the section secure initialization.

8.3.2 Input to Penetration Testing

147 An attacker may attempt to compromise the secure update process in order to execute unauthorized or malicious code. Potential scenarios include:

(A) An attacker attempts to load and execute unauthorized code.

(B) An attacker attempts to exploit input handling vulnerabilities in the update process to affect TOE integrity.

(C) An attacker attempts to bypass Secure Update using misconfiguration or improper enforcement of the security configuration.

(D) An attacker modifies firmware images (headers, certificate blocks, metadata, data, or signature) to attempt to bypass verification or affect the state after boot.

(E) An attacker attempts to exploit a vulnerability in the signature verification, including cryptographic algorithm strength, implementation and firmware format/error handling.

(F) An attacker attempts to exploit key handling vulnerabilities of the cryptographic verification mechanisms.

(G) An attacker attempts a replay attack of an older vulnerable firmware version.

(H) For the above attack paths, the attacker can combine logical attacks together with fault injection, possibly affecting internal processing and data, including modifying critical security values e.g. during read and processing, and the secure operation of verification mechanisms.

148 Based on these potential scenarios, some tests that could be carried out are:

- Attempt to load images with incorrect signatures or lower versions and verify that they are rejected.

Note that the same security mechanisms as for secure boot are applicable, see input to possible tests in the section of secure initialization.

8.4 Countermeasures and Physical Attacks

149 Anomaly detection and response describe the processes or algorithms that analyze the device input and output such as sensor data, as well as the software integrity during boot for abnormal events and, if required, trigger and execute a response. Typically, these actions encompass logging the anomaly and resetting the device to a known state, to prevent any further attempts of device manipulation.

150 Fault protection in the chip focuses on simple glitch protection for decision making processes critical to secure boot, debug, ARM JTAG opening, DFT JTAG opening, test mode entry and life cycle state interpretation.

151 The protection against physical attackers is achieved through several countermeasures.

(A) Glitch detection by the digital glitch detector (GDET) integrated in ELS. The GDET is enabled and configured by the boot ROM based on OTP information.

(B) Software flow protection built into the boot ROM design.

(C) The hardware code watchdog which provides secure counter and watchdog timer functions.

(D) Multi-bit hardened state machine protecting the critical interfaces (JTAG, CPU debugging, etc).

152 There are several mechanisms to consider when analysing countermeasures. As indicated earlier in the section of secure initialization, the boot sequence is protected though first logical detection and response mechanisms. The mechanisms are, as in part described earlier, the randomization of the boot flow, and double reading of the OTP, which is critical, and where there are additionally CRC32 error correcting codes. While CRC32 codes are error correcting and not cryptographically secure, they do provide additional protection against glitch and faults.

153 The GDET is initialized early in the boot process, and, importantly, before the firmware signature verification, therefore being able to detect attacks against the code authenticity and integrity in that stage for initialization, update and debug authentication, which also use the same mechanism.

154 Together, these mechanisms, logical and physical detection form a protection against faults in the boot process.

155 Further description of some security mechanisms related to the protection against physical attacks:

156 **Glitch Detector**

157 The glitch detector is a critical component in the TOE designs because it helps ensure reliability, security, and functional correctness. From the [AW/IW692_SAM]: “The S50 contains a digital glitch detector (GDET). The GDET gets enabled is after OTP read and before the FWDL begins. This means all of the security steps to validate the SB3 image are under GDET protection as is the JTAG enablement logic at the end of FWDL. The GDET clock is patch programmable in the step after OTP read but the chosen default value is what works for the SoC. It should be mentioned that the GDET OTP trim values are programmed during provisioning as are the DTRNG trim values (if applicable)”.

158 **Side-channel protection**

159 Given the security use case the manufacturer considers for RCP devices, which is only to support secure boot, there is no a particularly strong reason to have side-channel attack (SCA) countermeasures enabled in HW. The manufacturer states that the only asset that would need to be protected is the FW encryption key (derived from CUST_MK_SK), which they already protect against SCA (to some degree) by having the SB3.1 FW image format. However, side-channel attacks against the keys related to the confidentiality of the firmware is out of scope of the evaluation, since the confidentiality of the firmware is not in scope.

160 Besides AES decryption, ECDSA P-256 signature verification is used during secure boot. Since the manufacturer is handling public key during that operation, no SCA protection is needed by definition.

161 **Windowed watchdog timer (WWDT)**

162 From the [AW/IW692_SAM: “If the software fails to service the controller periodically within a programmable window, the watchdog resets the controller”. The main features are as follows:

(A) The chip will be reset if it is not periodically reloaded during the programmable time-out period.

(B) “Optional windowed operation requires reload to occur between a minimum and maximum time period, both programmable”.

(C) “Optional warning interrupt can be generated at a programmable time prior to watchdog time-out”.

(D) “Enabled by software but requires a hardware reset or a watchdog reset/interrupt to be disabled”.

(E) “If enabled, incorrect feed sequence causes reset or interrupt”.

(F) “Flag to indicate watchdog reset”.

163 **Code watchdog**

164 The Code watchdog provides two main mechanisms for detecting fault attacks, or execution of unexpected instruction sequences. The main features are extracted from the [AW/IW692_SAM]:

(A) “Secure counter (SEC_CNT): Detects altered software execution flow”.

- “The software loads the initial value to this counter, then issues ADD and SUB commands to increment/decrement the counter”.
- “Periodically, the software requests a secure counter value check by passing the expected value to the CWDOG using the STOP and RESTART commands”.
- A mismatch between the internal secure counter, and the value passed by the software, indicates that the expected execution flow is altered, possibly by a fault injection attack flow.

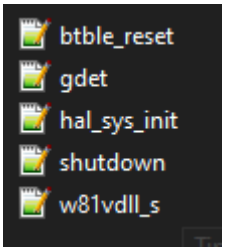
(B) “Instruction timer (INST_TIMER): Places a hard upper-limit on the interval between checks of the secure counter. The START command loads the internal decrementing secure counter. Before the counter underflows (reaches 0), a STOP or RESTART command must be executed, which forces the secure counter check”.

The combination of different countermeasures for the detection and response to fault injection attacks, both during the initialization and later in operational state significantly contributes to protect against fault injection attacks, by e.g. making them difficult to perform (e.g. randomization, multiple points of attacks and multiple bitflip patterns), and difficult to go undetected and without response, by e.g. glitch detector, comparison of values and the watchdog functionality guaranteeing the execution flow.

For side-channel attacks, fault injection attacks and hardware probing attacks, a specific analysis is presented in the sections corresponding to Secure Communication Support RCP – Network and Secure Communication Support RCP – Host, respectively.

8.4.1 Code implementation

165 In the case of the glitch detector, the evaluator can verify its use from the resources included in the "\AW693_security_related_source_code\GDET" directory. This directory has the following contents:



166

167 For example, the evaluator has found evidence of GDET initialization within the "hal_sys_init.c" file, shown below:

```
#if defined(GLITCH_DETECTOR)
    HAL_GET_CPU_ACCESS(ACC_REGION_SEC_IP, 0);
    if (WL_REGS32(CSS_STATUS_REG) &
        (CSS_STATUS_CSS_ERR_MASK | CSS_STATUS_GDET_IRQ_POS_MASK | CSS_STATUS_GDET_IRQ_NEG_MASK))
    {
        WL_WRITE_WORD(CSS_ERR_STATUS_CLR_REG, CSS_ERR_STATUS_CLR_ERR_CLR_MASK);
        WL_WRITE_WORD(CSS_GDET_EVTCNT_CLR_REG, CSS_GDET_EVTCNT_CLR_GDET_EVTCNT_CLR_MASK);
    }
    HAL_REL_CPU_ACCESS(ACC_REGION_SEC_IP);
#elif defined(GLITCH_DETECTOR_V2)
    if (HAL_SOC_INIT_FIRST_TIME())
    {
        // Approx 180 us time is required for Vcore to stable after changing it to SVC voltage
        // This data is provided by SiVal Team
        hal_WaitInUs(180);

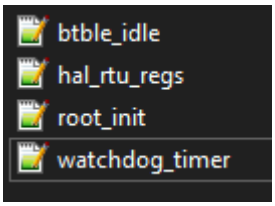
        /*Read GDET configuration from OTP*/
        if (hal_update_gdetCfgData() == FALSE)
            return;

        /* reload the GDET configurateion data */
        hal_loadGdetSVCConfig();
    }
#endif
```

168

169 In terms of side-channel attacks, the evaluator was able to find evidence of the use and implementation of the SB3 format for the images. Some evidence can be found in the subsection on secure initialization.

170 As for the watchdog code, the evaluator found the directory "\AW693_security_related_source_code\watchdog_timer", where the following content is found:



171

172 After reviewing the above content, the evaluator has found no clear evidence of the use of the Windowed watchdog timer (WWDT) in the code provided by the manufacturer. Therefore, the evaluator recommends the manufacturer

to verify the secure usage as described, and to validate this important feature by denial-of-service tests or similar in the penetration testing activities.

8.4.2 Input to Penetration Testing

173 An attacker may attempt to compromise the countermeasures and perform physical attacks with the goal of intercepting, modifying, impersonating, degrading, or extracting sensitive information. This includes firmware verification (secure boot, secure update or JTAG enablement) and OTP read, which is performed before GDET is activated. Several critical pieces of information can be affected during this operation (see below table extracted from [AW/IW692_SAM]:

(A) GDET configuration.

(B) TOE boot config, including security critical information for boot (CRC32 protection).

(C) DTRNG config

(D) Public key revocation information

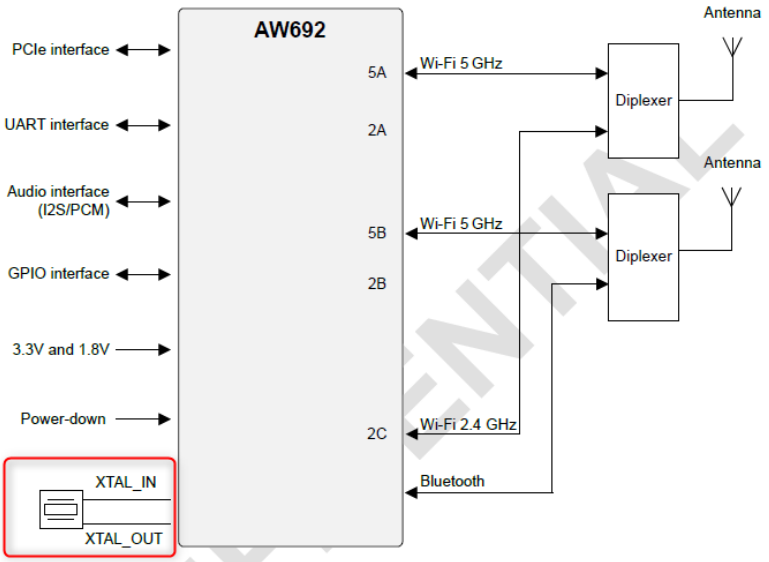
(E) Root hash (CRC32 protection)

(F) Security version number

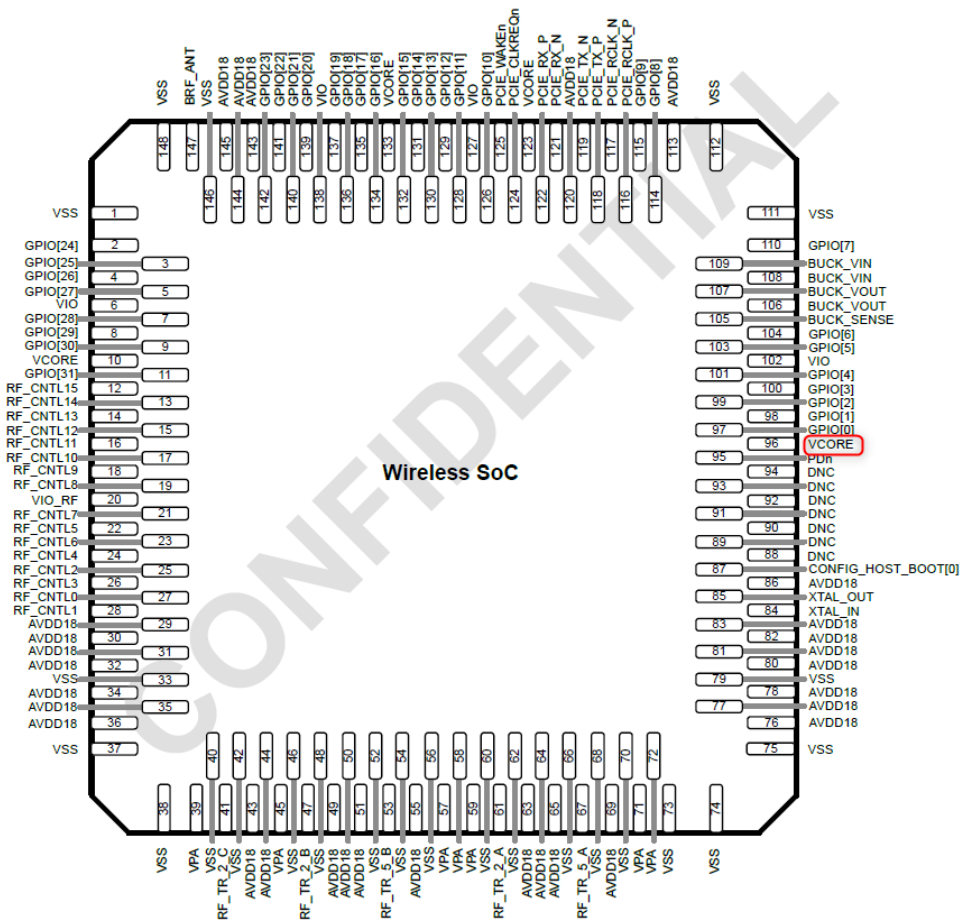
OTP Signatures			
Name	OTP signature	# of lines	Use
OTP_SIGN_PATCH_BLOCK_V3	0x09	Variable	Patch block
OTP_SIGN_HELPER_DATA	0x54	13	PUF config
OTP_SIGN_LCS	0x50	2	LCS
OTP_SIGN_SEBOOT_CONFIG	0x51	1	Flags for part
OTP_SIGN_ROOT_HASH	0x59	5	hash of 4 hashes
OTP_SIGN_GDET_CFG	0x58	5	GDET load data
OTP_SIGN_WRAPPED_KEY	0x56	7	Customer key
OTP_SIGN_UUID	0x57	3	UUID of the part
OTP_SIGN_PUBKEY_REVOKE	0x52	5	Revoke a key
OTP_SIGN_ANTI_ROLL_BACK	0x53	2	FW anti roll back
OTP_SIGN_RKEK	0x55	5	Used for ALT PUF
OTP_SIGN_DTRNG_CFG	0x5a	11	DTRNG load data
OTP_SIGN_RCAL	0x02	1	Analogue param

Potential scenarios include:

(A) Clock glitching: since an external crystal sources the TOE. An attacker could remove the clock from the board and place the clock source controlled by the attacker instead, which could introduce clock glitches. The working clock frequency is 40 MHz.



(B) Voltage glitching: an attacker could remove the decoupling capacitors routed to the power line and connect a voltage glitching tool.



(C) Electromagnetic Fault Injection: For this type of attack, no hardware modification is required. Therefore, an attacker could perform an automated campaign by varying the parameters of: pulse width, amplitude and delay of the injection signal.

(D) Attacks such as probing or component removal and replacement have been considered. Component removal and replacement is not applicable in this case, since all TOE components are either internal, or out of scope of the evaluation. Based on the functionality of the TOE and the security functions, the other interfaces are covered in the analysis of the secure communication support (TOE – Network, and TOE – Host), secure debug, secure initialization and secure update.

176 A conclusion is that EMFI and clock glitching are the least invasive fault injection methods for this TOE, and as such, motivated as first candidates for penetration testing using physical attacks. Voltage implies further physical modifications to the TOE and is more intrusive. Less intrusive have preference from an attacker perspective.

8.5 Secure Communication Support – RCP-Network

177 The platform supports various secure communication channels between the platform and external network devices, such as Wi-Fi and BT/BLE. These secure channels authenticate the communications endpoints and guarantee the confidentiality, integrity and authenticity of the data exchanged.

178 The protection is based on the correct implementation of the communications protocols and the security of the standardized protocols themselves. The RCP implements only the lower levels of the protocols, which means that the key negotiations and long term keys are never present or used in the RCP, but in the host. The RCP only get access to the ephemeral network credentials, encrypted from the host.

179 A detailed security analysis of the BT/BLE and Wi-Fi protocols is out of scope of the evaluation. However, publicly reported vulnerabilities and the implementation of the protocols are within the scope.

8.5.1 Attack scenarios involving physical attacks (SCA and FI) against Secure Communication Support – RCP – Network

180 The WiFi and BT/BLE communication protocols are implemented across the host and the RCP, where the RCP performs the per packet encryption and decryption, and the host performs the higher level session management, proving the necessary network credentials to the RCP (e.g. encryption keys for the channel).

181 This means that the encryption keys for the link layer security (WiFI/BT/BLE) are used in the encryption in the RCP and hence the risk of physical attacks against the key needs to be considered. The encryption key is crucial for the integrity and confidentiality of the transmitted data over the radio channel.

182 The following keys are assumed to be used in the RCP for encryption and potentially vulnerable to SCA and DFA attacks.

Protocol	Key
WiFi	Derived traffic key, TK
BT/BLE	Session key

183 Long term keys, such as passwords, are assessed as not transmitted to and stored in the RCP.

184 Common for these keys are that they are ephemeral and repeatedly renegotiated and changed. This effect the attack scenarios and their relevance.

8.5.1.1 Attack scenarios

185 In order to assess the applicability of an attack, the setting for the attack must be considered. There are differences between attacks in the field and attacks performed in a laboratory. In a laboratory attack, the attacker has access to controlled conditions and more advanced equipment than in the field. Consider the following observations.

(A) A laboratory attack is relevant if an attack can be performed in an initial phase, providing some asset, that can be used to later to attack the device when in the field.

(B) A laboratory attack is relevant if an attack can be performed in a later phase, where the device has been in the field, and past communication can be attacked based on assets that are extracted in this later phase.

(C) A laboratory environment is not applicable to field use, since a device working in the field would have to be transported to the laboratory, which has implications.

- The relatively short wireless connectivity range inhibits the laboratory to be the field environment, since the connectivity would be broken as the device is moved.
- Likely detected and could be prevented etc.

(D) A field environment attack does not have the same possibilities as a laboratory attack in terms of control of the environment and the type of equipment that can be used.

186 The fact that ephemeral keys are used for the WiFi and BT/BLE communication implies that laboratory attacks, independently of whether they are successful or not, are not meaningful as an initial phase or final phase to attack ephemeral keys used before or after the attack.

187 Relevant attacks on the ephemeral assets are hence restricted to field attacks. A field attack can be performed against a key that is in use. However, as soon as the ephemeral key is renegotiated, a new attack is needed. Since the ephemeral keys are regularly renegotiated, the consequence of a single attack is restricted to the communication of the session in question.

188 For continuous access, attacks need to be repeated for each session.

189 This is relevant since the attacks considered (Side-channel attacks and Fault injection attacks) requires physical access.

8.5.1.2 Physical attacks considerations

190 In the sequel, we analyze what the attacker would need in order to perform physical attacks in the field against one of the communicating devices.

8.5.1.2.1 Side-channel Attacks

191 The efficiency of SCA attacks on ephemeral keys is dependent on the leakage level, number of traces available and the physical equipment available.

(A) The number of traces is limited by the number of packets sent with the same key. The traffic is typically restricted on a wireless device, typically more so on BT/BLE devices than WiFi. The number of packets is limited by the re-key policy.

(B) Physical equipment depends on whether power analysis is used or electromagnetic analysis is used.

- For power analysis, typically, instrumentation and modifications of the hardware is needed. In order to provide high-quality measurements, the field environment is typically not ideal. An oscilloscope is needed for the measurements, and further a processing environment for the captured traces. While each attack may be similar, given the field environment conditions, it is likely that filtering of the signal is needed, and further processing such as aligning of the traces, to later perform e.g. a CPA or DPA attack. While in theory this can be automatized, it is likely that human interaction is needed.
  The measurements could be transmitted to somewhere else for remote processing.
  It is estimated that equipment of a non-negligible size is needed to be attached closely to the attacked device for as long as the attacker is interested in the communication.
  This means that the attack scenario is essentially implies an advanced hardware SCA trojan that is installed in the modified device, most likely needs human interaction for the SCA attacks to remain successful and furthermore additionally needs to record the wireless channel.

- For electromagnetic side-channel attacks, the scenario is similar, but instead of basing the analysis on the power variations, an EM probe is needed to be placed and maintained at a sufficiently good position, and the rest of the equipment, processing and analysis is the same. Compared to the power analysis approach, the difficulty is similar.

(C) For both power and EM SCA, if rekeying is sufficiently frequent in relation to the leakage, the attacks are infeasible.

(D) Both attacks require recording of the wireless channel, protocol analysis and decryption.

(E) Furthermore, there are additional difficulties regarding SCA attacks for the closed RCP.

- Since the RCP is closed, it is non-trivial to find and maintain a stable, suitable trigger point for capture of the traces, which raises the attack potential.
- The attacker is not assumed to have control of the plaintexts, key or ciphertext. In fact, there are several layers of protocol that need to be analyzed in order to match the measured traces to the actual output of the cipher. This most likely requires an additional stage in the SCA attack to identify and synchronize the ciphertexts with the traces. For unknown plaintexts, this may turn into a computational problem, where different synchronization points have to be used and tested, raising the attack potential.

8.5.1.2.2 Fault injection

192 Now consider fault injection attacks in the same setting.

(A) Fault injection in the field can be performed by different means, including voltage glitching, clock glitching and electromagnetic fault injection. In this case, high precision faults are required, that are introduced in the last rounds of encryption, in a way that is exploitable for differential fault injection. This type of faults can in theory be produced by both clock, voltage and electromagnetic fault injection.

(B) The equipment needed for the exploitation of the DFA attack is the injector of the fault, but also a way to produce a sufficiently reliable trigger. In this field scenario with a closed RCP, one way is to use a pattern-based trigger using an oscilloscope, and possibly filters, to achieve a trigger signal for the precise injection of the faults. It is assessed as very difficult to achieve a reliable and constant trigger in the field environment, unless lab equipment is used.

(C) Usage of voltage and clock glitching requires hardware modifications, while a probe is needed for electromagnetic fault injection. Likely, a high-end probe and high precision in positioning is needed for the precision required for successful DFA attack.

(D) The computations needed for key extraction can be performed remotely.

(E) It is noted that fault injection is invasive to some extent and the faults provoked can lead to reset of the device being attacked (or even to breaking the device), which would lead to renegotiation of the keys used. On the other hand, in theory, if a successful attack could be performed in a consistent and repeatable manner, a smaller number of traces is needed.

8.5.1.3 Attack potential

8.5.1.3.1 Fault injection

193 In the case of fault injection, there are countermeasures implemented in the RCP, which mitigates the attacks of fault injection, raising the difficulty of an attack from field conditions to a laboratory environment, leaving it out of scope by both non-applicability of the attack scenario and the high attack potential.

8.5.1.3.2 Side-channel analysis

194 The following attack potential estimation has been made under the assumption that the re-keying is not set to a sufficiently low value (which would make the attacks not applicable).

Factor	Identification	Exploitation
Elapsed Time	3 (< 1 month)	4 (< 1 week)*
Expertise	5 (Expert)	2 (Proficient)
Knowledge	0 (Public)	0 (Public)
Samples	0 (< 10)	0 (< 10)
Equipment	3 (Specialized)	4 (Specialized)**
Sum	**11**	**10**

195 * Physical adaptations to the target are needed, apart from the analysis. The time estimation here is not well adapted to this type of attack, since the attack needs to be concurrently be repeated, but not in all its parts. More than 1 day, but less than 1 week for the first exploitation, but after that, likely less than a day.
** The attack is estimated to specialized, since we foresee that a pattern-based trigger is needed, which we do not rate as standard. Also, in order to perform this attack repeatedly, specialized software development to automatize parts of the attacks are predicted.

196 Although SCA attacks could be theoretically possible, it is the opinion of the evaluators that they make no sense from an attack scenario point of view. Given the estimated attack potential (21), and the role of the RCP in a device, as essentially a smart antenna, and for the typical use cases for embedded systems, there are likely much more efficient ways for an attack to achieve its goal, which is the protected traffic on the wireless channel.

8.5.2 Summary

197 Based on the assessment of attack potentials and the attack scenarios, it is concluded that

198 For SCA

(A) if the re-keying is set to sufficiently frequent, SCA attacks are considered not applicable, and

(B) independently of the rekeying,

- field SCA attack scenarios are considered not applicable, and
- the attack potential is estimated out of scope for this TOE.

199 For FI

(A) the countermeasures for fault injection mitigate differential fault injection attacks and raises the attack potential to out of scope.

(B) field attack scenarios for DFA are considered not applicable.

8.5.3 Input to Penetration Testing

200 An attacker may attempt to compromise secure communication channels with the goal of intercepting, modifying, impersonating, degrading, or extracting sensitive information.

201 Potential scenarios include:

(A) An attacker attempts to intercept and decrypt traffic (disclosure).

(B) An attacker attempts to modify messages in transit to alter the behavior of the TOE or remote devices (modification).

(C) An attacker attempts to re-inject previously captured valid messages (replay).

(D) An attacker exploits bugs in Wi-Fi or BT/BLE implementations (negotiation, error handling, frame formats) to compromise security or cause unauthorized execution.

(E) An attacker attempts to force downgrade to weaker protocols/algorithms or to older vulnerable versions.

202 Based on these potential scenarios, some tests that could be carried out are:

- Attempt to capture traffic (sniffing) and verify that it cannot be decrypted without access to the protected cryptographic material (e.g., handshake capture and offline analysis). (ATE)
- Perform controlled Man-in-the-Middle (MITM) attacks to verify that endpoint authentication and key negotiation detect/reject impersonation attempts.
- Perform systematic fuzzing of the relevant protocol layers (Wi-Fi: WPA/WPA2/WPA3 in both AP and STA modes; Bluetooth Classic: Baseband and LMP; Bluetooth Low Energy: ATT/GATT) by sending malformed, unexpected, or random input. Validate that the TOE handles all inputs gracefully, without hanging, restarting, or executing unauthorized or arbitrary code, and that no sensitive data is leaked.

8.6 Secure Communication Support – RCP-Host

203 The TOE host interfaces provide message encryption and authentication mechanism for both Wi-Fi and Narrowband (NB). “This design utilizes a proprietary protocol (nanoTLS) based on the TLS 1.3 cryptographic handshake to establish a secure communication layer between the host CPU and device, via PCIe/SDIO.”

204 With nanoTLS-enabled, secure communication channel between the host CPU and device, either:

- All the configuration, control, event logging & tracing, stats counters and debug messages will be encapsulated into nanoTLS record.
- Selective types of messages which carry sensitive information (e.g. key material) that will be carried in nanoTLS record envelope format.

205 Secure Communication feature based on nanoTLS provides device authentication by the host of the RCP. The following properties are claimed:

- The host authenticates the device before issuing any further commands. Communication is only possible with a genuine NXP device.
- Data confidentiality (encryption) and authentication for all protected messages.
- Man-in-the-middle attack protection.
- Perfect forward secrecy (same as for TLS 1.3).

206 In this section, the proprietary nanoTLS protocol as basis for the secure communication between the host and the RCP is assessed. First, attack scenarios are identified, then an analysis of the different messages and stages of the protocol are treated, identifying properties that could be further analyzed or tested in the penetration testing. Finally, and concluding, the attack scenarios are mapped to the security properties of the protocol and implementation.

8.6.1 Attack scenarios

207 The use case of the secure channel between the host and the RCP (TOE) is to send the network credentials for WiFi and NB to the RCP from the host. These are the assets to protect. In the envisioned use cases, the host controls the RCP as a peripheral and authenticates the RCP in the session

establishment, hence ensuring that the network credentials are only transmitted on a secure channel to the authenticated RCP. Hence the channel is a wired, device internal channel using PCIe/SDIO.

208 In the following, a number of identified attack scenarios are presented. The security measures that mitigates the identified attacks are then analyzed further and presented in a later section.

(A) A Man-in-the-middle attempts to attack the confidentiality, integrity and authenticity of assets.

(B) An attacker attempts a SCA attack on the secure communication.

(C) An attacker attempts a DFA on the secure communication

(D) An attacker attempts replay attacks of the session establishment.

(E) An attacker attempts replay attacks on transmitted data or packets

(F) An attacker attempts an impersonation attack.

(G) An attacker attempts to modify packets and change the integrity of the data transmitted on the channel.

(H) An attacker attempts to exploit vulnerabilities in the implementation, for example due to insufficient input validation or incorrect handling of different values.

(I) An attacker attempts to downgrade the protocol or modify the configuration to an insecure mode of operation.

(J) An attacker attempts to exploit cryptographic use that is not best practice or legacy.

209 A property of the implemented secure channel is one-side authentication. Identified attack scenarios for this case is presented next, with a rationale on their applicability and relevance. While the lack of mutual authentication for the secure channel theoretically makes it possible to for an attacker to impersonate the host to the RCP, this scenario does not provide relevant attack scenarios. While many of the identified attack scenarios imply physical access and rewiring connection on a device, where the TOE is included, the attacks that specifically use the property of one-sided authentication have been further developed to make sure that the solution, that is less general, does not introduce any unforeseen vulnerabilities.

(A) Assume that the attacker impersonates the host and initiates a session establishment. The attacker can then produce and deliver network

credentials that are known to the attacker for usage of the RCP. However, in order for this attack to provide a meaningful result, the attacker must also provide corresponding network credentials to the other communicating party. On order to conduct this attack, the attacker needs

- Physical access to both parties
- Rewire/reconnect the interfaces to provide the communication on the PCIe/SDIO interface, which is assessed as complex, due to the speed and knowledge needed to perform this. In fact, detailed information on the communication protocol and hardware implementation of the interface is required.
- Place the network credentials on both (sending and receiving) communicating entities. Given that physical access to both communicating entities is needed, makes the attack complex and it is hard to see scenarios where this is a valid attack scenario.
- After placing planted network credentials, the attacker restores the device, where the TOE is one component, to the original physical setup, in both places.
- The restoration of the devices can´t lead to any reset of the network credentials after the attack, since then the communication would either be stopped, or new network credentials sent from the (authentic) host to the RCP. It is most likely the case that at least a reset of the RCP is needed, which would mitigate this outlined physical attack.

(B) Another attack scenario involves that the attacker tries to extract the network credentials from the RCP. The attacker would then need to

- Physical access to one of the communicating parties.
- Rewire/reconnect the interfaces to provide the communication on the PCIe/SDIO interface, which is assessed as complex, due to the speed and knowledge needed to perform this. In fact, detailed information on the communication protocol and hardware implementation of the interface is required.
- Use a vulnerability in the PCIe/SDIO interface of the RCP, or use valid host commands, to make the RCP export/extract the network credentials from the RCP.

- Use the network credentials to break the secure channel established. Depending on which protocol that is targeted, performing the attack may or may not involve setting up a MITM-attack.

(C) Yet another potential attack scenario, would be to instead of targeting the network credentials directly, target the private key of the RCP (TOE) to use it to attack the secure channel between the host and the RCP, for example to impersonate the RCP to the host. One way to do this would be through a combined physical and logical attack, similar to the previously described. The attacker impersonates the host to the RCP, and use an attack to extract the private key. This could be performed using some vulnerability during the session set-up, or after the channel is authenticated, or in an insecure channel. In a later stage, the attacker could use the private key to attack the network credentials from the host by impersonating the RCP. This attack requires e.g.

- Physical access to one of the communicating parties.
- Rewire/reconnect the interfaces to provide the communication on the PCIe/SDIO interface, which is assessed as complex, due to the speed and knowledge needed to perform this. In fact, detailed information on the communication protocol and hardware implementation of the interface is required.
- Use a vulnerability in the PCIe/SDIO interface of the RCP, or use valid host commands, to make the RCP export/extract the private key from the RCP.
- Use the private authentication key in an attack against the host. In the setting of the RCP as TOE, the interest would be in achieving the network credentials from the host, and then additionally use the network credentials to break the secure channel established with the other communicating device. Depending on which protocol that is targeted, performing the attack may or may not involve setting up a MITM-attack.

8.6.2 Analysis

210 The TOE uses a proprietary TLS1.3-based protocol (nanoTLS). In TLS terms, the Device acts as a Server, and the host acts as a client. The host is always trusted by the Device. There is no PKI involved.

211 The following cryptographic primitives are involved:

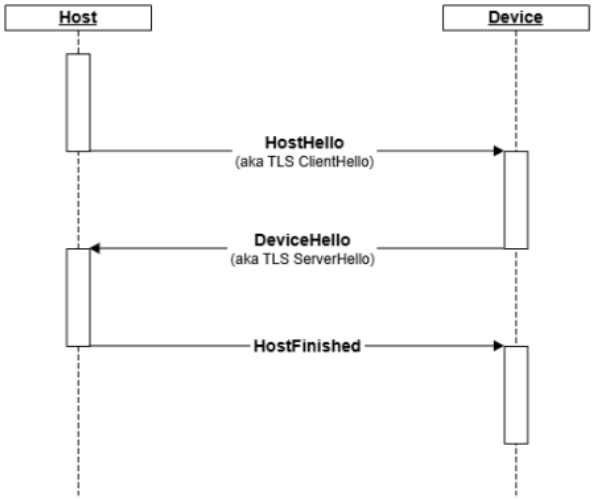
(A) Cryptographic Random Generator to produce “Hello.random“ and for ECC key generation.

(B) ECDHE for key exchange.

(C) ECDSA for authentication.

(D) SHA256 as a hash (HMAC, HKDF, signature generation).

(E) AES128-GCM for encryption and message authenticity.



212

213 The cryptographic primitives used (ECDHE, ECDSA, SHA256 and AES128-GCM) are assessed as best-practice.

214 The assumption “Host is always trusted by Device” has been further analysed to ensure that there are no relevant attacks that can be performed based on the one-sided authentication, see the section on attack scenarios.

8.6.2.1 HostHello

215 “Host uses signature_algorithm/key_exchange/cipher_suite/dev_auth_mode to specify which algorithms should be used by the device. No negotiation is allowed, host dictates the parameters. sig_alg field replaces Signature Algorithms TLS extension. Single algorithm is selected by host, there is no negotiation. This field defines the combination of the hash and digital signature to be used during the handshake. Currently only one combination is supported: ecdsa_secp256r1_sha256 (0x0403)”

216 **Input to penetration testing**: Verify that other options are not possible to use from the TOE side. This is to e.g. make sure that a session where different cipher options are provided will be rejected.

217 In RFC 8446, the stated extension data field of algorithms confirms the supported combination as a valid option:

```
Rescorla                    Standards Track                    [Page 41]

RFC 8446                          TLS                        August 2018


   The "extension_data" field of these extensions contains a
   SignatureSchemeList value:

      enum {
          /* RSASSA-PKCS1-v1_5 algorithms */
          rsa_pkcs1_sha256(0x0401),
          rsa_pkcs1_sha384(0x0501),
          rsa_pkcs1_sha512(0x0601),

          /* ECDSA algorithms */
          ecdsa_secp256r1_sha256(0x0403),
          ecdsa_secp384r1_sha384(0x0503),
          ecdsa_secp521r1_sha512(0x0603),

          /* RSASSA-PSS algorithms with public key OID rsaEncryption */
          rsa_pss_rsae_sha256(0x0804),
          rsa_pss_rsae_sha384(0x0805),
          rsa_pss_rsae_sha512(0x0806),

          /* EdDSA algorithms */
          ed25519(0x0807),
          ed448(0x0808),

          /* RSASSA-PSS algorithms with public key OID RSASSA-PSS */
          rsa_pss_pss_sha256(0x0809),
          rsa_pss_pss_sha384(0x080a),
          rsa_pss_pss_sha512(0x080b),

          /* Legacy algorithms */
          rsa_pkcs1_sha1(0x0201),
          ecdsa_sha1(0x0203),

          /* Reserved Code Points */
          private_use(0xFE00..0xFFFF),
          (0xFFFF)
      } SignatureScheme;
```

218

219 Similarly, for key_exchange_type, currently only one combination is supported ECDHE with secp256r1 curve (0x0017).

220 In RFC 8446, the stated key_exchange_type confirms the confirms the elliptic curve as a valid option:

```
4.2.7.  Supported Groups

When sent by the client, the "supported_groups" extension indicates
the named groups which the client supports for key exchange, ordered
from most preferred to least preferred.

Note: In versions of TLS prior to TLS 1.3, this extension was named
"elliptic_curves" and only contained elliptic curve groups.  See
[RFC8422] and [RFC7919].  This extension was also used to negotiate
ECDSA curves.  Signature algorithms are now negotiated independently
(see Section 4.2.3).

The "extension_data" field of this extension contains a
"NamedGroupList" value:

   enum {

       /* Elliptic Curve Groups (ECDHE) */
       secp256r1(0x0017), secp384r1(0x0018), secp521r1(0x0019),
       x25519(0x001D), x448(0x001E),

       /* Finite Field Groups (DHE) */
       ffdhe2048(0x0100), ffdhe3072(0x0101), ffdhe4096(0x0102),
       ffdhe6144(0x0103), ffdhe8192(0x0104),

       /* Reserved Code Points */
       ffdhe_private_use(0x01FC..0x01FF),
       ecdhe_private_use(0x FE00..0xFEFF),
       (0xFFFF)
   } NamedGroup;

   struct {
       NamedGroup named_group_list<2..2^16-1>;
   } NamedGroupList;
```

For the cipher suite, currently only one combination is supported: AES_128_GCM_SHA256 (0x1301)

This is a valid cipher suite in TLS 1.3:

```
B.4.  Cipher Suites

A symmetric cipher suite defines the pair of the AEAD algorithm and
hash algorithm to be used with HKDF.  Cipher suite names follow the
naming convention:

   CipherSuite TLS_AEAD_HASH = VALUE;

   +-----------+-----------------------------------------------+
   | Component | Contents                                      |
   +-----------+-----------------------------------------------+
   | TLS       | The string "TLS"                              |
   |           |                                               |
   | AEAD      | The AEAD algorithm used for record protection |
   |           |                                               |
   | HASH      | The hash algorithm used with HKDF             |
   |           |                                               |
   | VALUE     | The two-byte ID assigned for this cipher suite |
   +-----------+-----------------------------------------------+

This specification defines the following cipher suites for use with
TLS 1.3.

           +------------------------------+-------------+
           | Description                  | Value       |
           +------------------------------+-------------+
           | TLS_AES_128_GCM_SHA256       | {0x13,0x01} |
           |                              |             |
           | TLS_AES_256_GCM_SHA384       | {0x13,0x02} |
           |                              |             |
           | TLS_CHACHA20_POLY1305_SHA256 | {0x13,0x03} |
           |                              |             |
           | TLS_AES_128_CCM_SHA256       | {0x13,0x04} |
           |                              |             |
           | TLS_AES_128_CCM_8_SHA256     | {0x13,0x05} |
           +------------------------------+-------------+
```

225 **Input to penetration testing**: Verify that other options are not possible to use. Verify how different input values in the protocol are handled. Verify that the algorithms, parameters (curve) used can´t be affected.

226 It is stated that nonce is provided by the host: random 32 bytes of quality random data generated by Host, protects against replay attacks. The nonce should be sourced from secure random number generator.

227 In terms of the evaluation scope, this is an assumption. The assumption is not allowed to have negative influence on the TOE security. For the security model, with a trusted host, but unsecure channel, this is assessed to be feasible.

228 Key share for ECDH protocol.

229
- **key_share** - Q_h
  - Host's ECDHE public key derived from Host's ephemeral private key (randomly generated)
  - $Q_h = d_h \cdot G$ - EC scalar point multiplication
    - G - base point
    - d_h - Ephemeral Host private key (randomly generated)
    - Q_h - Ephemeral Host public key (key_share)

230 **Input to penetration testing**:

- The verification of the message by the TOE includes proper input validation. Negative test cases are discarded or handled securely.
- Verification that the point Q_h is on the NIST curve.
- The suggested algorithms etc. are as expected.

8.6.2.2 DeviceHello

231 DeviceHello is based on ServerHello. Only required fields and extensions are incorporated, everything else is removed and not supported. Device should either accept the Host cryptographic parameters from HostHello (sig_alg/key_exchange _type/cipher_suite) or reply with NACK. No negotiation is supported.

- **random**[32] (aka nonce) - Device's random 32 bytes of quality random data generated by Device
  - Protects against replay attack
  - Should be sourced from secure random number generator
- **key_share** - Q_d
  - Device's ECDHE public key derived from Device's ephemeral private key (randomly generated)
  - $Q_d = d_d \cdot G$ - EC scalar point multiplication
    - G - base point
    - d_d - Ephemeral Device private key (randomly generated)
    - Q_d - Ephemeral Device public key (key_share)
- **device_info** - device ID portion that is provided by the device FW
  - Transferred as-is to the host to help identifying the device (using key UUID)
  - Is acting as Certificate message like in RFC 7924 Transport Layer Security (TLS) Cached Information Extension, just by providing key ID, but not the certificate itself
- **device_handshake_sig** - signature generated by device over handshake context and is used for device authentication (using Device private key)
  - Follows CertificateVerify definition
  - should use signature specified in **HostHello.signature_algorithm** (currently only P-256 ECDSA is available)
- **device_finished** - Device Finished message
  - Computed the same way as defined in TLS
  - HMAC(finished_key) of handshake context up until the finished message itself
  - finished_key = HKDF-Expand-Label(device_hst_secret, "finished", "", Hash.length)

232

233 **Input to penetration testing**:

234 Verify in source code that the random nonce is in fact drawn from the secure random number generator.

235 Verify the outputs of several nonces as a sanity check that the nonces pass simple entropy tests.

8.6.2.2.1 Key share

236 Device's ECDHE public key derived from Device's ephemeral private key (randomly generated).

237 **Input to penetration testing**:

- The base point G can´t be affected by communication from the channel (this is hard coded and not suggested by the host)
- The base point G is on the NIST curve.
- The device´s ephemeral private key is randomly generated.
- The devices public key is correctly generated from the private key.

8.6.2.2.2 Device info

238 The device is identified by the key UUID of the public key, not by sending the certificate. This can be a sufficiently secure (lightweight) solution depending on the security model and assumptions on the environment (key UUID distribution and handling from the host).

239 Possible attack scenario:

- If the attacker has the private key of another certificate that is available to the host, this could potentially be used to impersonate the TOE to the host by the following procedure.
- The attacker performs exactly the same steps as the TOE, but instead uses the Key UUID of the key pair to which the attacker has the private key.
- The host, based on the received key ID, uses the indicated public key to verify the signature of the device (attacker), which is correct (but with another key pair).

240 Sending the certificate doesn´t resolve potential issue – unless PKI is used with CRLs.

241 If there is only one certificate, or a matching between the certificate and the process, the host can detect that another key UUID is used and use this as a protection mechanism. This seems to be the case (see later figure).

242 However, this potential attack scenario is against the host, which is out of scope for the current evaluation.

8.6.2.2.3 device_handshake_sig

243 Signature generated by the device over handshake context and is used for device authentication (using Device private key). Signing of all the previous messages detects possible modifications, and together with the nonces used, it introduces a replay protection against the session establishment. This is easy to see:

- Both parties have recorded their own nonce, and will use the values that they have sent and received. If any value was to be modified on the channel, they would get different values in the hash, and the signature verification would fail, assuming a proper signature verification is performed.
- The signature specified in HostHello.signature_algorithm (currently only P-256 ECDSA is available) should be used, which is best practice.

244 **Input to penetration testing:**

- Make sure that all messages and values over the channel are included in the hash for the signature.
- Verify that the signature verification is correct.

8.6.2.2.4 Device_finished

245 Device Finished message, computed the same way as defined in TLS HMAC(finished_key) of handshake context up until the finished message itself.

246 finished_key = HKDF-Expand-Label(device_hst_secret, "finished", "", Hash.length)

247 The whole handshake context is used in the hash, which is signed with HMAC using the derived secret key.

8.6.2.3 HostFinished

248 HostFinished is based on client Finished (sent by client in TLS). Only required fields and extensions are incorporated, everything else is removed and not supported.

249 host_finished - Host Finished message, is computed the same way as defined in TLS HMAC(finished_key) of handshake context up until the finished message itself.

250 finished_key = HKDF-Expand-Label(host_hst_secret, "finished", "", Hash.length)

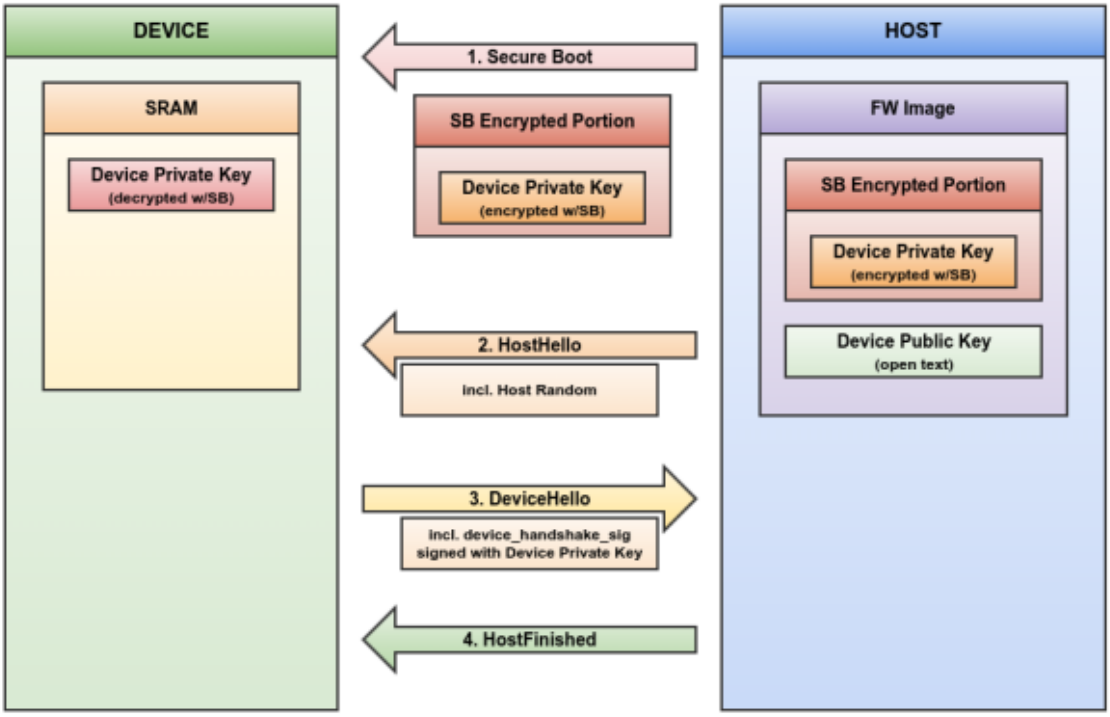
8.6.2.4 Conclusions

251 If there were any modifications to the communication, the device and host would obtain different hashes of the handshake context, and if related to the DH key exchange, different keys.

- The host can verify that the message context up to and including the information by the device, is the same between the host and the device, based on the signed DeviceHello message.
- The device can verify that the host has received the same values of the handshake as the device, based on the HostFinished message.
- As there is no signing providing authentication of the host to the device, the device can´t know whether the handshake is performed with the host, or with an attacker. However, in the envisioned use cases, this is assessed as a valid solution, no relevant attacks to the TOE have been identified based on this.
- It is noted that no processing of the DH secrets is exposed, other than through hash functions. It is also noted that the signature is applied to an internally generated hash value. Therefore, it seems that attempts

to e.g. use a combination of the signature and the DH key negotiation to gain information about private keys is infeasible.

252 The distribution of the signing key pair for the device is illustrated below.

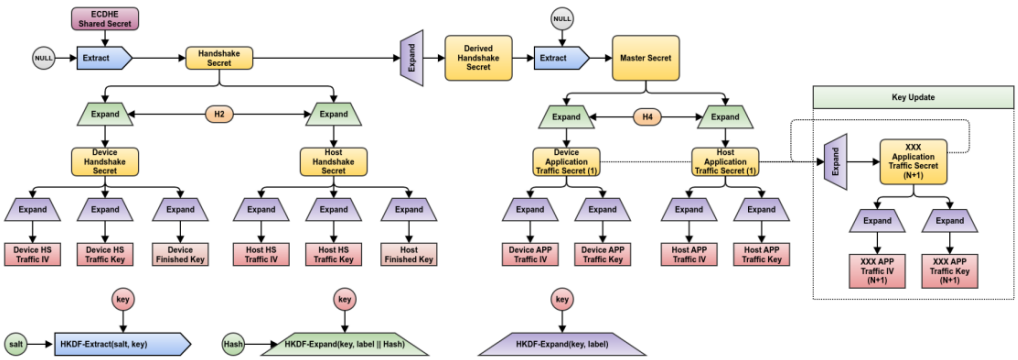


253

8.6.2.5 Handshake

254 In the following figure, the derivation of keys from the handshake is shown.

255 It is noted that HKDF is used, which is assumed to refer to RFC 5869, which is assessed as best practice.



256

257 It seems from the description of the “HostFinished”, that the information from the messages “HostHello” and “DeviceHello” is included, but not further information from the HostFinished, which could be included as well. This can

be seen in the description of the HostFinished, and in the figure of the derivation, where H2 is used, which is defined as

258 $$H2 = H(HostHello \| DeviceHello[: enc)$$

259 On the other hand, the following table of secrets, the Device/Host Finished key is indicated as being derived using the “HostFinished” context as well:

Host Application Traffic Secret	H4 = H(HostHello || DeviceHello || HostFinished)	"H AP TS"

Wireless Connectivity Engineering, BLE-SCE, NXP 356 Topic Page 6 of 10

COMPANY INTERNAL/PROPRIETARY NXP Security Architecture Manual -- Restricted/Confidenital

XXX Application Traffic Secret (N+1) (Device/Host key update)	""	"key update"
XXX XX Traffic Key (Device/Host HS/APP key)	""	"key"
XXX XX Traffic IV (Device/Host HS/APP IV)	""	"iv"
XXX Finished Key (Device/Host Finished Key)	""	"finished"

260

261 From the description of the key derivation, it is clear that IVs are also generated this way. This means that (random) information from both the host and the device is used, effectively ensuring that IV will have a sufficiently low risk of repetition, and making it unnecessary to explicitly send the IV over the channel.

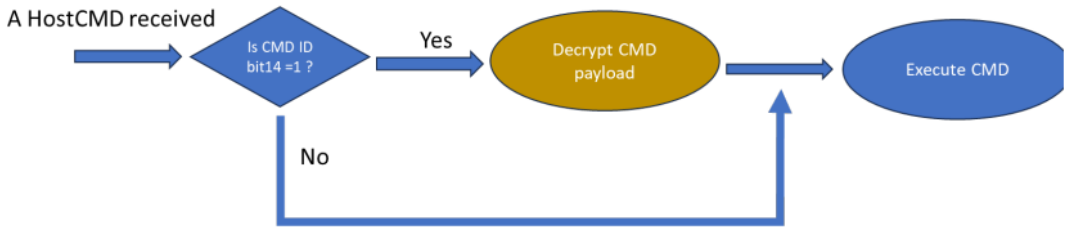
262 The authenticated encryption mechanism, AES-GCM, is very sensitive to the repetition of IV and key, since it generates the same key stream, and additionally the hash key can be attacked when repeated. The proposed generation algorithm ensures that the IV and key will not be repeated, unless the handshake ends up producing the same shared secret.

263 As previously analysed, the risk of handshake replay attacks is mitigated by the fact that both parties provide and verifies randomness to the handshake before key generation. The size of the nonces are large enough, leaving the risk of nonce collision to a secure level (2^128).

264 A nonce, based on the IV generated and sequence number (64 bits), is then used. Additional data, including the protocol version and the message length, are also included. The sequence numbers are maintained by both sides,

providing protection against replay attacks. The packet structure is described and assessed as best practice for encryption.

265 In the end, both secure and non-secure communication is permitted.



266

267 The channel can be used to send host commands to the TOE.

8.6.3 Input to Penetration Testing

(A) A Man-in-the-middle can perform attacks on confidentiality, integrity and authenticity of assets.

- As the analysis of the implemented nanoTLS demonstrates, a MITM attack is mitigated, since the RCP is authenticated by the host. A two-sided impersonation is needed in order to successfully perform a MITM-attack. See further the analysis of the protocol, where e.g. the complete session exchange is hashed and included as a confirmation.

(B) An attacker can perform a SCA attack on the secure communication.

- The SCA on the host with the objective to retrieve the encryption key is out of scope. Direct SCA (SPA) on the communication link to retrieve the network credentials is protected because of the encryption. Furthermore, the keys used are ephemeral and the considered case only use a small amount of data, which would give the attacker few samples to use for the attack.

(C) An attacker can perform a DFA on the secure communication

- The encryption of the network credentials is performed in host, which means that this type of attack is out of scope. Furthermore, the keys used are ephemeral.

(D) An attacker can perform replay attacks of the session establishment.

- This is mitigated by the protocol and the use of high-quality random numbers and the construction of the protocol. See further the analysis of the protocol.

(E) An attacker can perform replay attacks on transmitted data or packets

- Data transmitted are protected against replay attacks by the usage of protected sequence numbers. Both parties use counters to keep track of the sequence numbers to prevent replay attacks.

(F) An attacker can perform an impersonation attack.

- The host authenticates the RCP by the session establishment. The RCP does not authenticate the host during the session exchange, but the identified attacks using one-sided impersonation of the host to the RCP have been analyzed in separate and assessed as not relevant.

(G) An attacker can modify packets and change the integrity of the data transmitted on the channel.

- The session establishment is protected as analyzed in the analysis of the protocol.
- The packets after the session establishment is finished are protected by the authenticated encryption implemented, using AES-GCM.

(H) An attacker can exploit vulnerabilities in the implementation, for example due to insufficient input validation or incorrect handling of different values.

- Input to the penetration testing.

(I) An attacker can downgrade the protocol or modify the configuration to an insecure mode of operation.

- The crypto configuration is fixed and specified. No negotiation of the crypto used is possible.
- The correct implementation is input to the penetration testing to verify.

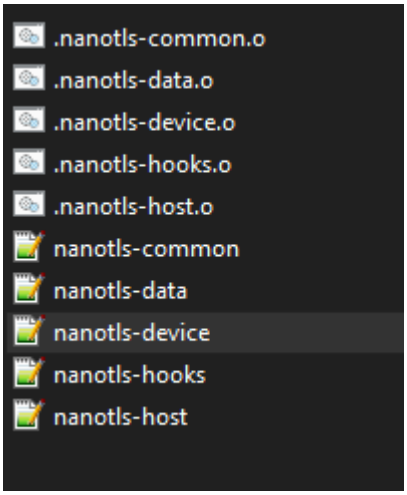
(J) An attacker can exploit cryptographic use that is not best practice or legacy.

- The selected algorithms are considered best-practice, and the protocol construction as well.

268 Therefore, the evaluator determines that there is no relevant attack scenario that is applicable in the penetration testing phase.

8.6.4 Code implementation

269 In terms of code, the evaluator has found different files related to nanotls within the "\AW693_security_related_source_code\bt_sec_if\" directory related to communications:



270

271 In files such as "nanotls-device.c", the tester found evidence of different parts of the protocol, such as the "HostHello" mentioned above:

```
enum ecode nanotls_device_host_hello_rcvd(struct nanotls_device_ctx *ctx,
                                          const struct nanotls_host_hello *msg)
{
    enum ecode ret = E_INVALID;

    if (ctx->state != NANOTLS_DEV_INIT)
        return nanotls_device_handle_error(ctx, E_INVALID);

    ret = nanotls_hdr_check(&msg->hdr, sizeof(*msg), NANOTLS_HOST_HELLO);
    if (ret != E_OK)
        return nanotls_device_handle_error(ctx, ret);

    /* Only support a single crypto algorithm combination for now */
    if (msg->sig_alg != NANOTLS_ECDSA_SECP256R1_SHA256)
        return nanotls_device_handle_error(ctx, E_INVALID);

    if (msg->key_exchange_type != NANOTLS_ECDHE_SECP256R1)
        return nanotls_device_handle_error(ctx, E_INVALID);

    if (msg->cipher_suite != NANOTLS_AES_128_GCM_SHA256)
        return nanotls_device_handle_error(ctx, E_INVALID);

    /* Feed the entire HostHello to the transcript hash context */
    nanotls_sha256_update(&ctx->handshake_hash, msg, sizeof(*msg));

    /* Save Host's ECDHE key share (ephemeral public key) */
    ctx->host_key_share = msg->host_key_share;

    ctx->state = NANOTLS_DEV_HELLO_RCVD;

    return E_OK;
}
```

272

273 Both in the previous screenshot, and in the code provided by the developer, the evaluator has contracted sufficient evidence of basic good practices such

as the minimum "input validation" to guarantee the expected behaviour of the routines.

8.7 System LifeCycle and Debug Authentication

274 Blackbird supports a security lifecycle state model that defines the functionality of the device, the accessibility of debug and test ports and the level of access to security assets. Mainly, the life-cycle management is handled by the Boot ROM together with the ELS in the immutable RoT, ensuring that the TOE is booted according to the correct life-cycle state.

275 Life-cycle state is monotonic, meaning it can only always be increased. The TOE has the following 5 life cycles:

Blank	OTP has no lifecycle programmed.
CM	Internal LCS - rom generated ENG keys
DM	Production LCS - rom generated Trust Provisioning keys
DD	Production LCS - OTP wrapped Trust Provisioning keys
FA	Failure analysis mode for customer returns - No keying.
CFA	Customer Failure analysis mode for SDKs - No keying.

276

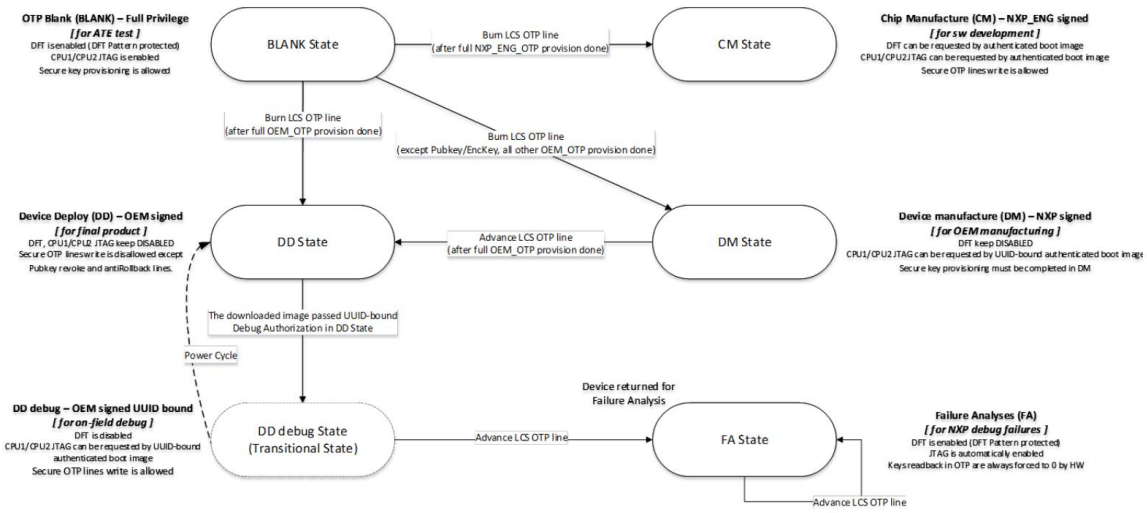
277 From an evaluation perspective, the TOE is in the DD state. If state transitions to previous states were possible, e.g. through a physical attack, that would violate the SFRs within scope. Furthermore, the secure debug, depending on the life-cycle state, is also within scope of the evaluation, and for the secure debug, the same comment holds with respect to attacks, either through changing the life-cycle or the verification mechanisms.

(A) “The CM life cycle state is exclusively used by NXP engineering and must never be presented to a customer. CM life cycle state uses internal ROM shares that are owned by NXP engineering.”

(B) The DM and DD life-cycle states are in the device manufacture and device deployment states. The main difference is in the Trust Provisioning Keys, ROM generated in DM and OTP wrapped in DD. Additionally, from the DD state, debug using a specific UUID-bound image can be performed.

(C) The FA life cycle state is for return to NXP to allow for examination of a failed device. This life cycle is reached from the DD.

(D) The CFA life cycle state allows a customer to develop their value add, using an SDK, w/o revealing NXP assets. This state is not included as part of the evaluation.



278 The life-cycle transitions are illustrated in the figure above.

279 Boot ROM acts as the main point of verification and determines the life-cycle state based on OTP fuse values. OTP fuses guarantee the irreversibility of transitions, mitigating rollback attacks of the states. Depending on the state, it enables or restricts functionality and access, such as debug interfaces.

280 The table below lists assets related to the keys used for firmware decryption in the device and explains how they are handled in various Lifecycle States. The keys and key shares mentioned are not part of the evaluation scope. The table is included for completeness, as, although not included in the evaluation scope, the encryption, and the secure key handling, can provide an additional security layer for an attacker, e.g. by making it more difficult to analyze the TOE for potential vulnerabilities.

Asset	Location	Blank	CM L0	CM L2	CM L3	DM	DD	DDD	FA
Manufacturing FW public key (Integrity)	SoC boot ROM	Not used Readable	Not used Readable	Same as CM L3	Used Readable	Used Readable	Not used Readable	Not used Readable	Not used Readable
ROM shares (Confidentiality, Integrity)	SoC boot ROM	Not used Readable	Not used Readable	Same as CM L3	Used Readable	Used Readable during boot, protected when JTAG is enabled	Not used Readable during boot, protected when JTAG is enabled	Not used Protected as JTAG is enabled	Not used Readable
RTL share (Confidentiality, Integrity)	ELS logic	Not used Not readable	Not used Not readable	Same as CM L3	Used Not readable	Used Not readable	Not used Not readable	Not used Not readable	Not used Not readable
UDF (Confidentiality, Integrity)	ELS logic	Not used Not readable	Not used Not readable	Same as CM L3	Used Not readable	Used Not readable	Might be used if PUF isn't working Not readable	Might be used if PUF isn't working Not readable	Not used Not readable
Customer Specific Share (Confidentiality, Integrity)	SRAM	N/A	N/A	Same as CM L3	N/A	Used Readable	N/A	N/A	N/A

281

Manufacturing FW encryption key (Confidentiality, Integrity)	ELS logic + SoC boot ROM	Not used Not readable	Not used Not readable	N/A	Used Not readable	Used Not readable	Not used Not readable	Not used Not readable	Not used Not readable
PUF key (Confidentiality, Integrity)	PUF logic (available in ELS as well during run-time)	N/A	Not used Not readable	Same as CM L3	Not used Not readable	Used Not readable	Used Not readable	Used Not readable	N/A PUF disabled
UDF-based device unique key (Confidentiality, Integrity)	ELS logic + OTP value	N/A	Not used Not readable	Same as CM L3	Not used Not readable	Might be used if PUF isn't working Not readable	Might be used if PUF isn't working Not readable	Might be used if PUF isn't working Not readable	N/A KEYPROV disabled
Key Wrapping Key (Confidentiality, Integrity)	PUF logic or UDF + OTP value (available in ELS as well during run-time)	N/A	Not used Not readable	Same as CM L3	Not used Not readable	Used Not readable	Used Not readable	Used Not readable	N/A PUF and KEYPROV disabled
Production FW public key (Integrity)	OTP	N/A	N/A	Same as CM L3	N/A	Not used Readable	Used Readable	Used Readable	Not used Readable

282

Production FW encryption key (Confidentiality, Integrity)	RFC3394 wrapped with KWK and stored in OTP. Available in ELS during run-time	N/A	N/A	Same as CM L3	N/A	Not used Not readable	Used Not readable	Used Not readable	Not used Not readable
Manufacturing FW (Confidentiality, Integrity, Authenticity)	SRAM			Same as CM L3	Used Readable		N/A	N/A	N/A
Production FW (Confidentiality, Integrity, Authenticity)	SRAM	N/A	N/A	Same as CM L3	N/A	N/A	Used Readable	Used Readable (DDD version)	N/A

283

284 Regarding the debug mechanism, the ARM JTAG is permitted to allow for invasive debug on both cores. “Normally, in the production setting, JTAG is closed by default.”

285 “For DD, an additional step is required for JTAG enablement. Not only must the encrypted image have the blanket JTAG enable but also it has to have a record that has a UUID in it. That 128bit UUID is burned into the device's OTP, only an image with both steps can unlock JTAG. The BootROM is suspicious of JTAG enablement and multiple checks are done to be sure that requested control flow asks are legitimate and not the result of glitches.”

286 The fact that the debug firmware is bound to a specific UUID, means that it can only be used for a specific selected device. The authenticity of opening the debug mechanism is based on the same signature verification as for secure update and secure initialization. Similarly, the security of replay of a debug opening firmware to another device, resides in the usage of device-unique UUID, and that the UUID in the debug firmware image is integrity protected, again by the same mechanism. While replay to other devices are mitigated by the UUID, it is noted that replay against the same device is possible by simply using the same package again.

8.7.1 Code implementation

287 Regarding the management of life cycles, the evaluator has found evidence of the definition in the BootROM code module, for example in the file: “\aw693w-a1-bootrom-src\common\cryptolib\src\comps\nxpClSb3\inc\nxpClSb3_Types.h" the evaluator has found the following evidences:

288
```
171 /*! @brief Enumeration for SoC Lifecycle. */
172 #define nxpClSb3_lc_nxpBlank (0xFFFF0000u)
173 #define nxpClSb3_lc_nxpFab (0xFFFE0001u)
174 #define nxpClSb3_lc_nxpDev (0xFF0300FCu)
175 #define nxpClSb3_lc_nxpProvisioned (0xFFFC0003u)
176 #define nxpClSb3_lc_oemOpen          (0xFFFC0003u)
177 #define nxpClSb3_lc_oemSecureWorld   (0xFFF80007u)
178 #define nxpClSb3_lc_oemClosed        (0xFFF0000Fu)
179 #define nxpClSb3_lc_oemLocked        (0xFF3000CFu)
180 #define nxpClSb3_lc_oemFieldReturn   (0xFFE0001Fu)
181 #define nxpClSb3_lc_nxpFieldReturn   (0xFF80007Fu)
182 #define nxpClSb3_lc_shredded         (0xFF0000FFu)
183 typedef uint32_t nxpClSb3_soc_lifecycle_t;
```

289 On the other hand, the evaluator has verified the use of cycles in different parts of the code. An example of this is in the file "\aw693w-a1-bootrom-src\common\seboot\src\seboot_SB3.c", specifically in the following image:

290
```
102     /* Only allowed to load in following LC */
103     if ((parms->soc_ecdsaNVM.soc_RoTNVM.soc_lifecycle == seboot_lc_Blank) ||
104         (parms->soc_ecdsaNVM.soc_RoTNVM.soc_lifecycle == seboot_lc_CM) ||
105         (parms->soc_ecdsaNVM.soc_RoTNVM.soc_lifecycle == seboot_lc_DM))
106     {
```

291 In the figure above the evaluator found a case in which content loading is only allowed if the TOE is in the "empty" state, CM or DM lifecycles.

292 As for the debug, the evaluator has been able to verify that part of the code is included within the BootROM code module. Among all the contents, the evaluator found evidence of the commands for the handling of the JTAG port in the following screenshot extracted from the file “\aw693w-a1-bootrom-src\common\include\auto_ciu_reg_define.h”:

293

```
#define CIU_JTAG_CIU_CMD0_JTAG_CIU_CMD_MASK        SHIFT0(0xffffffff)
#define CIU_JTAG_CIU_CMD0_JTAG_CIU_CMD_BASE        0
#define CIU_JTAG_CIU_CMD0_JTAG_CIU_CMD_HI          31
#define CIU_JTAG_CIU_CMD0_JTAG_CIU_CMD_LO          0
#define CIU_JTAG_CIU_CMD1       (0x90002604)
#define CIU_JTAG_CIU_CMD1_JTAG_CIU_CMD_MASK        SHIFT0(0xffffffff)
#define CIU_JTAG_CIU_CMD1_JTAG_CIU_CMD_BASE        0
#define CIU_JTAG_CIU_CMD1_JTAG_CIU_CMD_HI          31
#define CIU_JTAG_CIU_CMD1_JTAG_CIU_CMD_LO          0
#define CIU_JTAG_CIU_CMD2       (0x90002608)
#define CIU_JTAG_CIU_CMD2_JTAG_CIU_CMD_MASK        SHIFT0(0xffffffff)
#define CIU_JTAG_CIU_CMD2_JTAG_CIU_CMD_BASE        0
#define CIU_JTAG_CIU_CMD2_JTAG_CIU_CMD_HI          31
#define CIU_JTAG_CIU_CMD2_JTAG_CIU_CMD_LO          0
#define CIU_JTAG_CIU_CMD3       (0x9000260c)
#define CIU_JTAG_CIU_CMD3_JTAG_CIU_CMD_MASK        SHIFT0(0xffffffff)
#define CIU_JTAG_CIU_CMD3_JTAG_CIU_CMD_BASE        0
#define CIU_JTAG_CIU_CMD3_JTAG_CIU_CMD_HI          31
#define CIU_JTAG_CIU_CMD3_JTAG_CIU_CMD_LO          0
#define CIU_JTAG_CIU_CMD4       (0x90002610)
#define CIU_JTAG_CIU_CMD4_JTAG_CIU_CMD_MASK        SHIFT0(0xffffffff)
#define CIU_JTAG_CIU_CMD4_JTAG_CIU_CMD_BASE        0
#define CIU_JTAG_CIU_CMD4_JTAG_CIU_CMD_HI          31
#define CIU_JTAG_CIU_CMD4_JTAG_CIU_CMD_LO          0
#define CIU_JTAG_CIU_CMD5       (0x90002614)
#define CIU_JTAG_CIU_CMD5_JTAG_CIU_CMD_MASK        SHIFT0(0xffffffff)
#define CIU_JTAG_CIU_CMD5_JTAG_CIU_CMD_BASE        0
#define CIU_JTAG_CIU_CMD5_JTAG_CIU_CMD_HI          31
#define CIU_JTAG_CIU_CMD5_JTAG_CIU_CMD_LO          0
#define CIU_JTAG_CIU_CMD6       (0x90002618)
#define CIU_JTAG_CIU_CMD6_JTAG_CIU_CMD_MASK        SHIFT0(0xffffffff)
#define CIU_JTAG_CIU_CMD6_JTAG_CIU_CMD_BASE        0
#define CIU_JTAG_CIU_CMD6_JTAG_CIU_CMD_HI          31
#define CIU_JTAG_CIU_CMD6_JTAG_CIU_CMD_LO          0
#define CIU_JTAG_CIU_CMD7       (0x9000261c)
#define CIU_JTAG_CIU_CMD7_JTAG_CIU_CMD_MASK        SHIFT0(0xffffffff)
#define CIU_JTAG_CIU_CMD7_JTAG_CIU_CMD_BASE        0
#define CIU_JTAG_CIU_CMD7_JTAG_CIU_CMD_HI          31
#define CIU_JTAG_CIU_CMD7_JTAG_CIU_CMD_LO          0
```

8.7.2 Input to Penetration Testing

294 There are several possible attack scenarios that are relevant to the secure debug mechanism:

(A) An attacker attempts to access debug leveraging potential vulnerabilities based on misconfiguration or improper enforcement of the security and life-cycle configuration.

(B) An attacker with access to debug attempts to exploit a vulnerability gain information and access that should not be available in debug. This includes any data stored by the application.

(C) An attacker attempts to exploit vulnerabilities in the debug authentication mechanism to gain access. This includes the signature verification, including cryptographic algorithm strength, implementation and usage, key handling, replay attacks and error handling.

(D) An attacker attempts to use the debug image corresponding to one device to access debug for another device.

(E) An attacker attempts to reverse the life-cycle of the TOE process in order to gain debug access.

(F) For the above attack paths, the attacker can combine logical attacks together with fault injection, possibly affecting internal processing and data, including modifying critical security values e.g. during read and processing, and the secure operation of verification mechanisms.

295 Based on these potential scenarios, some tests that could be carried out are:

- Attempt to access debug with an incorrect debug image and verify that it is rejected. This can be performed by both logical and physical means, targeting different parts of the verification, the life-cycle state or other configuration data.

Note that the similar security mechanisms are used as for secure boot and update, see further input to possible tests in the section of secure initialization.

9 Penetration Testing

9.1 Introduction

296 This table lists the tests and their corresponding categories that will be carried out in the penetration testing.

Pub. Vuln. Id.	Test Plan Reference	Category	Description	Estimated Attack Potential
N/A	AVA_SW_TEST.01	[SW] Software Attacks	Source code vulnerability analysis	12
N/A	AVA_HW_TEST.01	[INT] External Interface Attacks [POW] Fault Injection Attacks	Unlock the debug port with fault injection attack	20
N/A	AVA_COM_TEST.01	[COM] Communication-based Attacks	Fuzzing attack against Bluetooth interface	15
N/A	AVA_COM_TEST.02	[COM] Communication-based Attacks	Fuzzing attack against Wi-Fi interface	15
N/A	AVA_SB_TEST.01	[LOG] Logical Security Attacks	Test the secure boot and update binary verification mechanism	15
N/A	AVA_SB_TEST.02	[LOG] Logical Security Attacks [POW] Fault Injection Attacks	Bypass the signature verification mechanism with fault injection	20
N/A	AVA_SB_TEST.03	[LOG] Logical Security Attacks	Test the availability support mechanism	N/A
N/A	AVA_SU_TEST.01	[LOG] Logical Security Attacks [POW] Fault Injection Attacks	Bypass the anti-rollback mechanism with fault injection	20

297 In the selection of tests for the penetration testing, the relevance of the attack scenarios for the TOE scope has been considered, as well as the evaluation scope in terms of security functions.

298 Remote attacks are among the most important attacks for the TOE to resist. Therefore, testing of the communication protocols using fuzzing tools are included. Note also that the functional part of the testing has included the verification of authentication and encryption of the wireless communication interfaces of the TOE.

299 Furthermore, key security functions of the TOE are the secure initialization and the secure update mechanisms. Penetration tests against these functions are included based on their importance to security of the TOE. The suggested tests include both logical attacks and physical attacks in the form of fault injection attacks.

300 Debugging is an important attack surface and therefore penetration tests against the debug interface are suggested using fault injection attacks.

301 Side-channel attacks have been considered in the attack scenarios and also for the penetration testing, given that the scope of evaluation includes side-channel attacks. However, confidentiality of firmware (update) is not included in the scope of evaluation. Also, the secure communication security function is a natural candidate for side-channel attacks. However, it is concluded from the attack scenarios that the secure communication is executed using ephemeral cryptographic keys, which makes the attack scenarios irrelevant. Repeated side-channel attacks and continuous access to the TOE would be required by the attacker, in which case the attacker could just as well attack the plaintext directly from the e.g. the host or by other means. Also, the low number of traces that could be available for the attacker is likely makes a successful side-channel attack improbable. These attack scenarios are included and further analysed in the vulnerability analysis chapter for the corresponding security function: Secure Communication Support RCP-Network.

302 Apart from the identified attack scenarios, further analysis of source code, involving static analysis and code review of selected parts of the code is proposed. This is proposed to be included as a general task (static code analysis) and as a complement for the specific tests, e.g. secure boot and update and to some degree for the implementation of secure communications.

The plan for the penetration testing is the set of suggested tests. Based on potential findings, potential deviations in the effort and complexity of executing the tests, test cases may be omitted, extended or added.

9.1.1 Software interfaces

9.1.1.1 Work unit AVA_SW_TEST.01

AVA_SW_TEST.01: Source code analysis

Objective: Source code vulnerability analysis.

Description: The analysis of the source code is important, as the different security functionalities (secure boot, update, communication) uses the security resources of the TOE, and these are critical because they could lead to unwanted behavior, or unauthorized use of the TOE's security functions.

Examples:

- Use of specific tools to perform code analysis to find vulnerabilities (e.g. Synopsys tools environment).
- Manual analysis of selected parts of the source code.

Rating:

Factor	Identification	Exploitation
Elapsed time	≤ one week (2)	≤ one day (3)
Expertise	Proficient (2)	Layman (0)
Knowledge of the TOE	Public (0)	Public (0)
Access to TOE	< 10 samples (0)	< 10 samples (0)
Equipment	Specialized (3)	Standard (2)
Open Samples	Public (0)	NA
Sub-total	7	5
Total	**12**	

Motivation: There should be no publicly known vulnerabilities in used libraries. The software interfaces are crucial for the isolation properties of the TOE from a potential remote attacker getting access to e.g. the non-secure environment.

9.1.2 Hardware interfaces

9.1.2.1 Work unit AVA_HW_TEST.01

AVA_HW_TEST.01: Debug access port

Objective: Unlock the debug port with fault injection attack

Description: Perform a test campaign based on fault injection attacks to cause

unexpected behaviour during configuration and verification of access to the debug port and thus unlock the access to the port that should be closed. Therefore, partial or full control of the debug port could be obtained.

This test will support in assessing the protection of the chip against fault attacks, which implicitly tests the different countermeasures, including the glitch detector and the logical detection mechanisms. Furthermore, applying the fault injection tests, the TOEs ability to recover from faults is verified, which is related to the partial testing of the security function Availability Support (Watchdog).

Examples:

- The test campaign can be performed with clock glitching or electromagnetic fault injection. Single fault injection is considered.
- Based on a reference signal or trigger, perform a scan through the TOE and modify different parameters of the attack such as amplitude, probe, position of probe and timing for EMFI, and e.g. timing and clock shape for clock glitching.

Factor	Identification	Exploitation
Elapsed time	≤ one month (3)	≤ one week (4)
Expertise	Proficient (2)	Proficient (2)
Knowledge of the TOE	Restricted (2)	Restricted (2)
Access to TOE	< 10 samples (0)	< 10 samples (0)
Equipment	Specialized (3)	Standard (2)
Open Samples	Public (0)	NA
Sub-total	10	10
Total	**20**	

Motivation: The debug ports are a typical part of the attack surface for an attacker with physical access to the device.

9.1.3 Communication interfaces

9.1.3.1 Work unit AVA_COM_TEST.01

AVA_COM_TEST.01: Bluetooth fuzzing

Objective: Fuzzing attack against Bluetooth interface

Description: It is proposed to use a Bluetooth fuzzing tool to test the Bluetooth implementation of the TOE.

Examples:

- Use of specific tools to perform fuzzing against Bluetooth interface.

Rating:

Factor	Identification	Exploitation
Elapsed time	≤ one week (2)	≤ one week (4)
Expertise	Proficient (2)	Proficient (2)
Knowledge of the TOE	Public (0)	Public (0)
Access to TOE	< 10 samples (0)	< 10 samples (0)
Equipment	Specialized (3)	Standard (2)
Open Samples	Public (0)	NA
Sub-total	7	8
Total	**15**	

Motivation: Wireless communication interfaces are the main elements that an attacker can access the TOE if the protocol implementation is not secure, even interrupting communications between the TOE and the rest of the elements of the communications channel.

9.1.3.2 Work unit AVA_COM_TEST.02

AVA_COM_TEST.02: Wi-Fi fuzzing

Objective: Fuzzing attack against Wi-Fi interface

Description: It is proposed to use a Wi-Fi fuzzing tool to test the Wi-Fi implementation of the TOE.

Examples:

- Use of specific tools to perform fuzzing against Wi-Fi interface.

Rating:

Factor	Identification	Exploitation
Elapsed time	≤ one week (2)	≤ one week (4)
Expertise	Proficient (2)	Proficient (2)
Knowledge of the TOE	Public (0)	Public (0)
Access to TOE	< 10 samples (0)	< 10 samples (0)
Equipment	Specialized (3)	Standard (2)
Open Samples	Public (0)	NA
Sub-total	7	8
Total	**15**	

Motivation: Wireless communication interfaces are the main elements that an

attacker can access the TOE if the protocol implementation is not secure, even interrupting communications between the TOE and the rest of the elements of the communications channel.

9.1.4 Secure initialization of platform

9.1.4.1 Work unit AVA_SB_TEST.01

AVA_SB_TEST.01: Modified bootloader binary
Objective: Test the secure boot and update binary verification mechanism.
Description: The secure boot mechanism can be tested by making different types of modifications in different fields of the firmware binary. These tests should all give the results that the signature verification fails, if no earlier test or input validation mechanism is implemented. This test case will also include the secure update mechanism since both share the verification logic of the binary. In addition, the evaluator will pay attention to the anti-rollback mechanism. This is complemented by the analysis of the relevant parts of the source code. The analysis is hence based on the restricted information and the source code provided by the manufacturer, but to perform an attack, no restricted information is needed in principle.
Examples: Based on the SB3.1 binary format, some binary modifications can be made: • Length extension. • Signature. • The firmware version field. • Removing parts of the binary. • Different fields of the binary. **Note**: The modification of the binary signature could be addressed in the ATE totally or partially.

Factor	Identification	Exploitation
Elapsed time	≤ one week (2)	≤ one day (3)
Expertise	Proficient (2)	Proficient (2)
Knowledge of the TOE	Restricted (2)	Public (0)
Access to TOE	< 10 samples (0)	< 10 samples (0)
Equipment	Standard (1)	Standard (2)
Open Samples	Public (0)	NA
Sub-total	7	7
Total	**14**	

Motivation: Secure boot is a fundamental root-of-trust security property.
In the case of secure update, the anti-rollback is a security property to prevent downgrade attacks to earlier, potentially vulnerable versions.

9.1.4.2 Work unit AVA_SB_TEST.02

AVA_SB_TEST.02: Bootloader bypass

Objective: Bypass the signature verification mechanism with fault injection.

Description: Perform a test campaign based on fault injection attacks to cause unexpected behaviour during the boot process, including, but not restricted to, the firmware signature verification.

This test will support in assessing the protection of the chip against fault attacks, which implicitly tests the different countermeasures, including the glitch detector. Furthermore, applying the fault injection tests, the TOEs ability to recover from faults is verified, which is related to the partial testing of the security function Availability Support (Watchdog).

Examples:

- The test campaign can be performed with clock glitching or electromagnetic fault injection. Single fault injection is considered.
- Based on a reference signal or trigger, perform a scan through the TOE and modify different parameters of the attack such as amplitude, probe, position of probe and timing for EMFI, and e.g. timing and clock shape for clock glitching.

Factor	Identification	Exploitation
Elapsed time	≤ one month (3)	≤ one week (4)
Expertise	Proficient (2)	Proficient (2)
Knowledge of the TOE	Restricted (2)	Public (0)
Access to TOE	< 10 samples (0)	< 10 samples (0)
Equipment	Specialized (3)	Standard (2)
Open Samples	Public (0)	NA
Sub-total	10	8
Total	**18**	

Motivation: Secure boot is a fundamental security root-of-trust property.

9.1.4.3 Work unit AVA_SB_TEST.03

AVA_SB_TEST.03: Availability support mechanism

Objective: Test the watchdog mechanism

Description: The availability of software execution is supported by two watchdog mechanisms:

- A watchdog timer to reset the platform if the software does not respond within a time window.
- A hardware code watchdog for fault attacks or execution of unexpected instruction sequences.

The availability is implicitly tested during the AVA_COM_TEST.01 and AVA_COM_TEST.02

Examples:
TOE's availability will be tested by verifying it is not disrupted during the fuzzing campaign performed in the Communication tests.

Motivation: Availability support mechanism guarantee that TOE's main functionalities are not disrupted, building a robust security system.

9.1.4.4 Work unit AVA_SB_TEST.04

AVA_SB_TEST.04: OTP read attack

Objective: Modify the OTP read before GDET is activated

Description: Perform a test campaign based on fault injection attacks to cause

unexpected behaviour during the OTP read. According to the [AW/IW692_SAM]: "GDET cannot be enabled right after PoR as it required some part tuned values to be loaded from OTP into the GDET. When the GDET gets enables is after OTP read and before the FWDL begins."

This test will support in assessing the protection of the chip against fault attacks, which implicitly tests the different countermeasures against OTP read attacks, including the randomization and logical detection mechanisms in the boot process.

Examples:

- The test campaign can be performed with clock glitching or electromagnetic fault injection. Single fault injection is considered.
- Based on a reference signal or trigger, perform a scan through the TOE and modify different parameters of the attack such as amplitude, probe, position of probe and timing for EMFI, and e.g. timing and clock shape for clock glitching.

Factor	**Identification**	**Exploitation**
Elapsed time	≤ one month (3)	≤ one week (4)
Expertise	Proficient (2)	Proficient (2)
Knowledge of the TOE	Restricted (2)	Public (0)
Access to TOE	< 10 samples (0)	< 10 samples (0)
Equipment	Specialized (3)	Standard (2)
Open Samples	Public (0)	NA
Sub-total	10	8
Total	**18**	

Motivation: OTP read is part of the secure boot flow.

9.1.5 Secure update of platform

9.1.5.1 Work unit AVA_SU_TEST.01

AVA_SU_TEST.01: Anti-rollback bypass

Objective: Bypass the anti-rollback mechanism with fault injection.

Description: In this test, the firmware version defined in the binary with SB3.1 format is lower than the accepted version by the TOE. Given that the TOE is expected to not accept the modified firmware update, a test campaign based on fault injection attacks to cause unexpected behaviour during the update process is

proposed, specifically to bypass the verification of the firmware version. A successful fault injection attack against the anti-rollback mechanism could lead to a downgrade attack.

This test will support in assessing the protection of the chip against fault attacks, which implicitly tests the different countermeasures, including the glitch detector. Furthermore, applying the fault injection tests, the TOEs ability to recover from faults is verified, which is related to the partial testing of the security function Availability Support (Watchdog).

Examples:

- The test campaign can be performed with clock glitching or electromagnetic fault injection. Single fault injection is considered.
- Based on a reference signal or trigger, perform a scan through the TOE and modify different parameters of the attack such as amplitude, probe, position of probe and timing for EMFI, and e.g. timing and clock shape for clock glitching.

Factor	**Identification**	**Exploitation**
Elapsed time	≤ one month (3)	≤ one week (4)
Expertise	Proficient (2)	Proficient (2)
Knowledge of the TOE	Restricted (2)	Public (0)
Access to TOE	< 10 samples (0)	< 10 samples (0)
Equipment	Specialized (3)	Standard (2)
Open Samples	Public (0)	NA
Sub-total	10	8
Total	**18**	

Motivation: Anti-rollback is a security property to prevent downgrade attacks to earlier, potentially vulnerable versions.

10 Evidence List

1 The evidences are listed in the document [NXP_AW692-EL_v1.1].